UNIVERSIDADE DE SANTIAGO DE COMPOSTELA



ESCOLA TÉCNICA SUPERIOR DE ENXEÑARÍA

Estudio y mejora del rendimiento de modelos Octree para búsqueda de vecinos en nubes de puntos 3D.

Autor/a:
Pablo Díaz Viñambres

Tutores:
Francisco Manuel Fernández Rivera
Miguel Yermo García

Grado en Ingeniería Informática

Junio 2025

Traballo de Fin de Grao presentado na Escola Técnica Superior de Enxeñaría da Universidade de Santiago de Compostela para a obtención do Grao en Enxeñaría Informática

Resumen

En este trabajo se presenta un enfoque para la búsqueda eficiente de vecinos en nubes de puntos 3D, en especial las obtenidas mediante tecnología LiDAR. El método propuesto consiste en un reordenamiento espacial de la nube fundamentado en las Curvas de Llenado del Espacio (SFCs), junto a implementaciones eficientes de búsqueda basadas en Octrees. Se explora cómo las SFCs de Morton y Hilbert pueden optimizar la organización de los datos tridimensionales, mejorando el rendimiento de las consultas de vecindad y la construcción de estructuras sobre la nube. Se proponen y evalúan diversas variantes del Octree, analizando su impacto en la eficiencia y escalabilidad del método propuesto. Los resultados experimentales demuestran que el reordenamiento mediante SFCs y el uso de algoritmos de búsqueda sobre Octrees especializados mejoran significativamente el acceso a datos espaciales, ofreciendo una solución robusta para aplicaciones que requieren procesamiento rápido de grandes conjuntos de puntos 3D. Una parte de este trabajo ha sido presentado en las XXXV Jornadas de Paralelismo de SARTECO.

Índice general

1. Introducción **1**

2. Revisión del estado del arte **3**

3. Metodología **7**

3.1. Nubes de puntos LiDAR 7

3.2. Reordenamiento de nubes de puntos 9

3.2.1. Curva de Morton 11

3.2.2. Curva de Hilbert 12

3.2.3. Proceso de reordenamiento de la nube 13

3.3. Estructuras Octree 15

3.3.1. Octrees basados en punteros 16

3.3.2. Octrees lineales 18

3.4. Algoritmos de búsqueda de vecinos optimizados 25

3.4.1. Inserción directa de puntos con octante contenido en kernel 25

3.4.2. Estructura basada en rangos para el resultado 27

3.4.3. Búsquedas de vecinos aproximadas 28

4. Pruebas **31**

4.1. Entorno de pruebas y paralelización 31
4.2. Métodos de búsqueda exactos 34
4.3. Comparación con otras estructuras 36
4.4. Búsquedas aproximadas 40
4.5. Eficiencia en el paralelismo 41
4.6. Reordenamiento y construcción de *Octrees* 42

5. Discusión de los resultados **45**

5.1. Conclusiones obtenidas 45
5.2. Comparación con el estado del arte 46

6. Conclusiones y posibles ampliaciones **47**

A. Manuales técnicos **49**

A.1. Archivos fuente y headers principales 49
A.2. Instrucciones de compilación 50
A.3. Script utilizado en las pruebas 51

B. Manuales de usuario **53**

C. Licencia **57**

Bibliografía **59**

Índice de figuras

3.1. SFCs bidimensionales. 11

3.2. Curva de Morton tridimensional sobre S_1, S_2 y S_3. 12

3.3. Curva de Hilbert tridimensional sobre S_1, S_2 y S_3. 13

3.4. Reordenamientos de las nube de puntos *Lille_0* (10 millones de puntos, terrestre) y *5080_54400* (12 millones de puntos, aérea). La escala de color marca el orden en memoria en el que se encuentran los puntos. Por visibilidad, solo se muestran 100 000 puntos, uniformemente muestreados. 14

3.5. Bordes de los octantes de un *Octree* sobre la nube de puntos *Lille_0*, reordenada mediante la curva de Hilbert. 17

3.6. Visualización de la nube a través de cada paso del proceso de subdivisión en cuadrantes. 21

3.7. Parte interna del *Octree* lineal construido sobre la nube del Ejemplo 3.1. 22

3.8. Búsqueda de vecinos aproximada sobre un centro de la nube *Lille_0*, con *kernel* esférico de radio 3 y varias tolerancias. Los valores negativos de tolerancia se corresponden con búsquedas de cota inferior. 28

4.1. Diagrama de la arquitectura de la máquina NUMA. 33

4.2. Tiempo de ejecución para 5000 búsquedas de vecinos con centros aleatorios en la nube *Paris_Luxembourg_6*. Algoritmo *neighborsPrune* usado en el *Octree* lineal. 34

4.3. Nube de puntos en escala *log-log* mostrando la relación entre el tiempo de 5000 búsquedas aleatorias y μ en varias nubes y configuraciones. Algoritmo *neighborsPrune* utilizado en el *Octree* lineal. 35

4.4. Tiempos acumulados para 5000 búsquedas de vecinos con centros aleatorios en la nube *Paris_Luxembourg_6* para varios algoritmos de búsqueda. La nube fue reordenada mediante la SFC de Hilbert. 36

4.5. Nube de puntos en escala *log-log* mostrando la relación entre el tiempo de 5000 búsquedas con centros aleatorios y los distintos algoritmos de búsqueda utilizados. La nube fue reordenada mediante la SFC de Morton. 37

4.6. Comparación del rendimiento de distintas estructuras en búsquedas aleatorias en $\mathcal{N}_{Sphere}$. La nube fue reordenada mediante la SFC de Hilbert. 39

4.7. Comparación en la aproximación de búsquedas completas en la nube *Lille_0*. La nube fue ordenada mediante la SFC de Hilbert. . 40

4.8. Matrices de eficiencia para la paralelización de búsquedas en el dataset *Lille_0*. Algoritmo *neighborsPrune* usado en el *Octree* lineal. 41

Índice de cuadros

3.1. Estructuras de los puntos de la nube. Los primeros 4 campos contienen la información geométrica y el identificador del punto (estructura `Point`), mientras que los demás consisten en metadatos provenientes del escáner LiDAR (estructura `PointMetadata`). . . 9

3.2. Resumen de cada uno de los componentes de un *cornerstone-octree*. 25

4.1. Combinaciones de *Octree* y SFC utilizadas en las pruebas. 32

4.2. Algoritmos de búsqueda utilizados en las pruebas. 32

4.3. Tiempos de ejecución de búsquedas de vecinos sobre todos los puntos de la nube con *Sphere kernel* y con SFC de Morton. 38

4.4. Comparación de tiempos de ejecución promedio (en segundos) sobre varios datasets por rangos de μ. Kernel esférico y reordenación de Hilbert usados. 39

4.5. Tiempos de cálculo de la codificación y reordenamiento sobre varias nubes y SFCs. 42

4.6. Tiempos de construcción de los *Octrees* sobre varias nubes en segundos. Notación: t_h tiempo de construcción de las hojas, t_i de la parte interna y t_t tiempo total, en segundos. 43

B.1. Opciones principales del programa 54

B.2. Opciones adicionales del programa 55

Capítulo 1

Introducción

El cálculo de vecindarios es una operación fundamental en numerosos algoritmos de procesamiento geométrico y aprendizaje automático. Tareas como la clusterización, la segmentación, la extracción de propiedades o el minado de datos en nubes de puntos se basan en la habilidad de poder encontrar puntos próximos a un centro de forma eficiente.

Dado un conjunto o nube de puntos tridimensionales $\mathcal{P} \subset \mathbb{R}^3$ y un centro $c \in \mathcal{P}$, una búsqueda de vecinos busca encontrar aquellos puntos de $\mathcal{P}$ que están próximos a c, con el objetivo de encontrar cierta estructura local sobre la nube. Existen dos formas de calcular vecindarios: las búsquedas k-NN, en las que se hallan los k puntos más cercanos a c, y las *búsquedas de radio fijo*, en las que nos centraremos durante este trabajo. Estas últimas buscan todos los puntos $q \in \mathcal{P}$ tales que la distancia de c a q sea menor que un cierto radio $r \in \mathbb{R}$. Por supuesto, esta distancia puede ser considerada en diferentes normas o dimensiones.

En nuestro caso, las nubes de puntos se obtienen a partir de sensores LiDAR (*Light Detection And Ranging*), que llevan a cabo la reconstrucción de entornos 3D emitiendo una gran cantidad de rayos láser sobre una escena real de estudio. La medición temporal entre la emisión y captura de cada rayo permite medir la distancia y dirección desde el sensor hasta la superficie en la que rebota. Junto con otros mecanismos como la Unidad de Control, Monitoreo y Grabación (CMRU) y el sistema de Posicionamiento y Orientación (POS), se puede determinar la posición tridimensional de cada uno de estos rebotes con alta precisión, creando así una nube de puntos en la que aparecen cada uno de los millones de impactos de los haces emitidos.

Esta tecnología, aparecida por primer lugar en los años 60, se ha extendido en los últimos años. Hoy en día existen diversos tipos de sensores disponibles comer-

cialmente a un precio cada vez más asequible. Entre otros tipos, existen sensores terrestres (TLS y MLS), que suelen montarse sobre un trípode y sensores aéreos, montados en drones (ULS) o aeroplanos (ALS), empleados en capturas de grandes áreas de terreno. Debido al gran número de aplicaciones y la disponibilidad de la tecnología, LiDAR se ha convertido en una herramienta imprescindible en el campo de la teledetección.

Tras el proceso de captura, los retornos del láser se sincronizan y procesan junto a otros datos provenientes de otros dispositivos del escáner y finalmente se almacenan en un formato adecuado. Existen varios estándares para el almacenamiento de nubes de puntos LiDAR, pero a lo largo de este trabajo nos centraremos en el más ampliamente usado, el formato LAS [30]. Los archivos binarios `.las` recogen la coordenadas geométricas (x, y, z) y otras características capturadas por el sensor como la intensidad del rayo recibido o el número de retornos. También es común encontrar datos de color para cada punto si existe una cámara RGB acoplada al sensor. Los archivos `.las` se dividen en 4 secciones: encabezado, registros de longitud variable (VLR), registro de puntos y registros de longitud variable extendidos (EVLR).

Es de especial interés el formato del registro de puntos. Con el propósito de adaptarse al gran rango de usos y formas de capturar las nubes de puntos LiDAR, existen múltiples variantes para este, recogidas en el estándar LAS 1.4 [30]. En nuestros desarrollos, representamos los puntos en una estructura de datos siguiendo el formato *"Point Data Record Format 2"* del estándar, con algunas variaciones como la representación de las coordenadas y la intensidad de cada punto en punto flotante de doble precisión, además de un identificador con el orden de lectura de cada punto.

A lo largo de este trabajo, buscaremos optimizar las búsquedas de vecinos mediante dos mecanismos complementarios. La reordenación mediante curvas de llenado del espacio (SFCs) servirá para mejorar la localidad de la nube, permitiendo que los puntos próximos a otro en el espacio también se encuentren próximos en memoria. Por otro lado, construiremos *Octrees* sobre las nubes de puntos, que aportaran una visión del espacio jerárquica en la que es posible desarrollar algoritmos optimizados de búsqueda. Como veremos, las SFCs y los *Octrees* están profundamente relacionados, y es posible aprovecharse de este hecho para lograr una gran eficiencia.

Todo el código empleado en nuestros desarrollos, tanto la implementación de los algoritmos, como los scripts que generan las visualizaciones, está disponible en el repositorio de GitHub [13]. En el Anexo A detallamos la estructura del programa, damos instrucciones de compilación y el script utilizado para ejecutar el programa principal. En el Anexo B enumeramos las opciones del programa.

Capítulo 2

Revisión del estado del arte

Debido a la naturaleza no estructurada e irregular de las nubes de puntos LiDAR, es necesario desarrollar estructuras de datos y algoritmos que den soporte eficiente a las búsquedas, tanto en radio fijo como k-NN, para cualquier tamaño de nube, sensor utilizado o superficie capturada. En la literatura, existen numerosas propuestas buscando condensar la geometría de las nubes de puntos de la forma más eficiente posible.

En este trabajo, emplearemos dos construcciones fundamentales: los *Octrees* y las curvas de llenado del espacio (SFCs). El uso de ambos mecanismos ha experimentado un crecimiento significativo en los últimos años, siendo utilizados repetidamente para gestionar y analizar nubes de puntos 3D. Los *Octrees*, que dividen recursivamente el espacio en ocho octantes, se benefician de las SFCs al mapear datos multidimensionales en un orden unidimensional, optimizando aplicaciones como la compresión, el indexado y las consultas espaciales, incluyendo búsquedas de vecinos más cercanos. Entre las SFCs, destacan las curvas de Morton y de Hilbert, que tienen fortalezas complementarias, y su adopción depende de los requisitos específicos de cada aplicación.

Los códigos de Morton [26], conocidos por su simplicidad y rapidez, son ampliamente utilizados en estructuras *Octree* para tareas de indexado y compresión. Su integración en *Octrees* ha sido clave para mejorar la eficiencia, como se manifiesta en el trabajo de Behley et al., donde se emplean para ordenar la nube de puntos y reducir la necesidad de recorrer el árbol en toda su profundidad durante búsquedas de vecindades, superando el rendimiento de librerías populares tales como PCL y FLANN [4]. En el ámbito de la compresión, Cao et al. proponen un método basado en *autoencoders* que transforma nubes de puntos en códigos de Morton utilizando un *Octree* lineal, inspirándose en su efectividad en tareas de análisis visual y compresión de imágenes [8]. Además, Yang et al. destacan cómo

los códigos de Morton facilitan estrategias de particionado adaptativas, asegurando la localidad de los datos y mejorando el rendimiento en bases de datos no relacionales [37]. En softwares de nubes de puntos como CloudCompare, se utilizan codificaciones tipo Morton para búsquedas binarias rápidas y barridos espaciales, mientras que Liu et al. las aplican en una estructura híbrida de *Octree* y 3D R*-Tree para optimizar consultas de vecinos más cercanos [35].

Las curvas de Hilbert han ganado cada vez más popularidad debido a su superioridad, con respecto a los códigos de Morton, para preservar la localidad espacial al indexar nubes de puntos tridimensionales [6]. Algunos estudios han demostrado la superioridad de las curvas de Hilbert frente a las de Morton a la hora de mantener la localidad durante el análisis de nubes de puntos [33]. Además, algunas investigaciones recientes destacan la aplicabilidad de las curvas de Hilbert para mejorar la eficiencia de la compresión de datos [29], [11]. En [12] se propone el uso de curvas de Hilbert para reducir la dimensionalidad de una nube de puntos tridimensional, y así poder aplicar operaciones directamente en dos dimensiones, que de otra manera serían demasiado costosas. A pesar de ser una curva más complicada que la de Morton, existen varios algoritmos optimizados para su cálculo [24], [25], haciendo apropiado su uso incluso en contextos de alto rendimiento. En general, los códigos de Morton y de Hilbert ofrecen una forma de trasladar un espacio tridimensional a un espacio unidimensional, lo cuál es inmediatamente aplicable al almacenamiento de una nube de puntos en memoria.

Sin embargo, hasta donde sabemos, la aplicación de las curvas de Morton y Hilbert para la búsqueda de vecindades en nubes de puntos 3D obtenidas con sensores LiDAR no ha sido estudiada en profundidad. En [15], se presenta una librería escrita en C++ para manejar nubes de puntos masivas en paralelo utilizando curvas de llenado de espacio. Sin embargo, la librería está orientada para su uso con bases de datos relacionales. En relación a las consultas espaciales, los autores demuestran que el uso de curvas de llenado de espacio, en especial la de Hilbert, supera los métodos clásicos de consulta espacial (como el B-Tree) cuando el número de datos a manejar es de unos pocos cientos de millones.

Keller et al. [23] presentan un método de construcción de *Octree* basado en arrays lineales construidos a partir de las codificaciones SFC (de Morton o de Hilbert), para su utilización en el contexto de simulaciones de partículas en entornos distribuidos. La variante de *Octree* desarrollada para estas simulaciones, conocida como *cornerstone-octree*, será la estructura base en el desarrollo de nuestros métodos de búsqueda en radio fijo optimizados. Esta estructura será ampliada y equipada con datos adicionales para acelerar los algoritmos clásicos de búsqueda de vecinos. Para ello, adaptaremos algunas de las ideas expuestas por Behley et al. [4], tales como la poda en búsquedas en profundidad sobre el árbol, y las comprobaciones geométricas optimizadas para comprobar si cada nodo del árbol

se encuentra dentro o interseca el volumen de búsqueda.

Finalmente, existen métodos alternativos a los *Octrees* para las búsquedas de vecindades. Cuando el radio de búsqueda se conoce de antemano, son también populares las estructuras basadas en cuadrículas fijas, que pueden ser implementadas en GPUs por su naturaleza altamente paralela [19]. Por otro lado, los *kd*-trees son utilizados en la aceleración de búsquedas k-NN [5]. En particular, destacamos el proyecto FLANN (Fast Library for Approximate Nearest Neighbors) [27] para la aproximación de estas búsquedas k-NN mediante *kd*-trees. Esta librería está integrada en PCL [32] y proporciona una interfaz para búsquedas con radio fijo, lo que la posiciona como una referencia adicional para evaluar nuestros desarrollos.

Capítulo 3

Metodología

En este capítulo, detallaremos las ideas principales detrás de los algoritmos y estructuras de datos sobre las que realizaremos nuestras pruebas. Comenzaremos exponiendo el formato en el que se almacenan las nubes de puntos LiDAR y cómo reordenarlas para mejorar la localidad espacial en su almacenamiento en memoria. A continuación, describiremos las distintas estructuras *Octree* sobre las que trabajaremos. Finalmente, discutiremos varias implementaciones de búsqueda de vecinos con radio fijo, cuyo rendimiento mediremos en el capítulo de pruebas.

3.1. Nubes de puntos LiDAR

Comenzamos definiendo formalmente las nubes de puntos objeto de nuestro estudio.

Definición 3.1. Sea $\mathcal{D} = \{p_1, p_2 \ldots, p_N\} \in \mathbb{R}^{N\times m}$ un conjunto de N puntos con m características. Cada punto p_i tiene coordenadas geométricas $(x_i, y_i, z_i) \in \mathbb{R}^3$. Llamamos *nube de puntos (3-dimensional)* al conjunto

$$\mathcal{P} = \{(x_i, y_i, z_i) \in \mathbb{R}^3 : i = 1, 2, \ldots, N\} \in \mathbb{R}^{N\times 3}.$$

Definimos también las esquinas superior e inferior p_{min} y p_{max} de la nube:

$$p_{min} = (x_{min}, y_{min}, z_{min}) = \left(\min_{1\le i\le N} x_i, \min_{1\le i\le N} y_i, \min_{1\le i\le N} z_i\right),$$

$$p_{max} = (x_{max}, y_{max}, z_{max}) = \left(\max_{1\le i\le N} x_i, \max_{1\le i\le N} y_i, \max_{1\le i\le N} z_i\right),$$

y su *bounding box* $\mathcal{B} = [x_{min}, x_{max}) \times [y_{min}, y_{max}) \times [z_{min} \times z_{max})$.

Emplearemos las librerías de código abierto *laslib* y *laszip* [21, 20] para la lectura, escritura y compresión de estas nubes de puntos en el formato LAS. Cabe añadir que, por razones de eficiencia y precisión, las coordenadas almacenadas en los archivos LAS se encuentran en formato entero $(x_{\text{point}}, y_{\text{point}}, z_{\text{point}}) \in \mathbb{Z}^3$, pero se convierten a punto flotante durante su lectura multiplicando y añadiendo ciertos factores de escala $(x_{\text{scale}}, y_{\text{scale}}, z_{\text{scale}})$ y desplazamiento $(x_{\text{offset}}, y_{\text{offset}}, z_{\text{offset}})$ recogidos en el encabezado del archivo, como se indica en la siguiente ecuación:

$$\begin{cases} x = x_{\text{point}} \cdot x_{\text{scale}} + x_{\text{offset}}, \\ y = y_{\text{point}} \cdot y_{\text{scale}} + y_{\text{offset}}, \\ z = z_{\text{point}} \cdot z_{\text{scale}} + z_{\text{offset}}. \end{cases} \tag{3.1}$$

Tras la lectura del archivo de entrada `.las` y la conversión a punto flotante de (3.1), almacenamos la nube en dos arrays paralelos, uno con los primeros 4 campos del cuadro 3.1, conteniendo la información geométrica necesaria para las búsquedas (`std::vector<Point>`) y otro conteniendo los demás campos (`std::vector<PointMetadata>`). Ambas estructuras `Point` y `PointMetadata` se pueden alinear mediante relleno a 32 bytes, pudiendo almacenar así dos elementos en cada línea caché, cuyo tamaño es de 64 bytes en la mayoría de arquitecturas.

A lo largo de este trabajo, utilizaremos el siguiente conjunto heterogéneo de datasets de nubes LiDAR para nuestras pruebas y visualizaciones, todos ellos populares entre investigadores del ámbito:

- *Paris-Lille-3D* [31]: Este dataset mapea dos calles de París. Posee una alta densidad y está capturado mediante tecnología de LiDAR terrestre, con sensor MLS. Trabajaremos con segmentos de 10 millones de puntos como *Lille_0* o *Paris_Luxembourg_6*.

- *DALES* [34]: Se trata de un dataset de LiDAR aéreo dividido en celdas disjuntas de aproximadamente 10 millones de puntos. Su densidad es menor, por lo que, en promedio, se encontrarán menos puntos en las vecindades.

- *Semantic3D* [17]: Es un dataset LiDAR de alta densidad capturado con sensor fijo sobre trípode. Utilizamos las nubes *bildstein_station1* (30 millones de puntos) y *sg27_station8* (430 millones de puntos) en nuestro análisis.

- *Speulderbos* [7]: Este dataset está tomado mediante un sensor LiDAR montado en dron y sobre un trípode en tierra. Presenta la densidad más alta de todos y consta de aproximadamente 721 millones de puntos.

Cuadro 3.1: Estructuras de los puntos de la nube. Los primeros 4 campos contienen la información geométrica y el identificador del punto (estructura `Point`), mientras que los demás consisten en metadatos provenientes del escáner LiDAR (estructura `PointMetadata`).

Campo	Descripción	Tipo	Tamaño
x	Coordenada eje X	double	8 bytes
y	Coordenada eje Y	double	8 bytes
z	Coordenada eje Z	double	8 bytes
id	Identificador	unsigned long	8 bytes
I	Intensidad	double	8 bytes
psId	ID de Fuente del Punto	unsigned short	2 bytes
r	Canal de color rojo	unsigned short	2 bytes
g	Canal de color verde	unsigned short	2 bytes
b	Canal de color azul	unsigned short	2 bytes
ud	Datos de usuario	unsigned char	1 byte
classification	Clasificación del punto	unsigned char	1 byte
sar	Ángulo de escaneo	char	1 byte
rn	Número de retornos	char	3 bits
nor	Número de retornos del pulso	char	3 bits
dir	Dirección de escaneo	char	1 bit
edge	Límite de la línea de vuelo	char	1 bit

3.2. Reordenamiento de nubes de puntos

En esta sección se desarrollará un método para optimizar la *localidad o proximidad espacial* de nubes de puntos LiDAR. Con este término, nos referimos a la propiedad de que puntos próximos en el espacio tridimensional estén próximos en memoria. Como veremos más adelante, nuestros métodos de búsqueda de vecinos se basan en la habilidad de poder encontrar puntos en una nube cercanos a uno dado de forma eficiente. Para esto, una alta localidad espacial es beneficiosa, ya que provocará un uso más eficiente de la jerarquía de memoria. Particularmente, aumentará el número de aciertos en la caché, lo cual evitará costosas búsquedas en RAM.

Nuestra propuesta se basa en el reordenamiento de la nube en memoria [1], cuyos puntos se encuentran originalmente en el orden de escaneo del sensor LiDAR. Para ello, necesitamos encontrar una transformación desde el espacio tridimensio-

[1]Con el término "reordenar la nube" nos referimos al reordenamiento de los vectores en los que se almacenan los datos, tanto las coordenadas geométricas como los metadatos LiDAR. Aunque en las búsquedas de vecinos solo se usan las coordenadas, en aplicaciones de más alto nivel es esencial contar con las mediciones de intensidad, color y ángulos del sensor LiDAR.

nal de los datos hasta el espacio unidimensional de memoria en el que se almacena la nube [2]. En primer lugar, es conveniente *discretizar* la nube, es decir, convertir sus coordenadas a números enteros.

Fijamos entonces un nivel de subdivisión $L \in \mathbb{N}$ y tomamos la rejilla $S_L = [0, 2^L) \times [0, 2^L) \times [0, 2^L) \cap \mathbb{N}^3$, $|S_L| = 2^{3L}$. Buscamos discretizar la bounding box $\mathcal{B}$ de forma que a cada punto $p \in \mathcal{P}$ le corresponda una celda $c \in S_L$. Para ello, se realizan los siguientes pasos:

1. Trasladamos y escalamos las coordenadas de cada uno de los puntos de la nube al cubo $[0, 2^L) \times [0, 2^L) \times [0, 2^L) \subset \mathbb{R}^3$.
2. Convertimos cada uno de los puntos transformados en elementos de S_L mediante la función suelo. De esta forma, obtenemos unas coordenadas $(\mathtt{x}, \mathtt{y}, \mathtt{z})$ en forma de enteros de L bits, relativas a la bounding box de la nube. Por ejemplo, la esquina inferior de $\mathcal{B}$ se convierte en el $(0, 0, 0)$:

$$(x_{min}, y_{min}, z_{min}) \to (\underbrace{000\ldots0}_{L \text{ bits}}, \underbrace{000\ldots0}_{L \text{ bits}}, \underbrace{000\ldots0}_{L \text{ bits}}).$$

Como $|S_L| = 2^{3L}$, es conveniente representar cada una de las celdas como un número entero o *código* de $3L$ bits. Encontrar formas de asignar cada uno de estos códigos a cada una de las celdas de forma que se maximice la localidad espacial es la cuestión principal de esta sección.

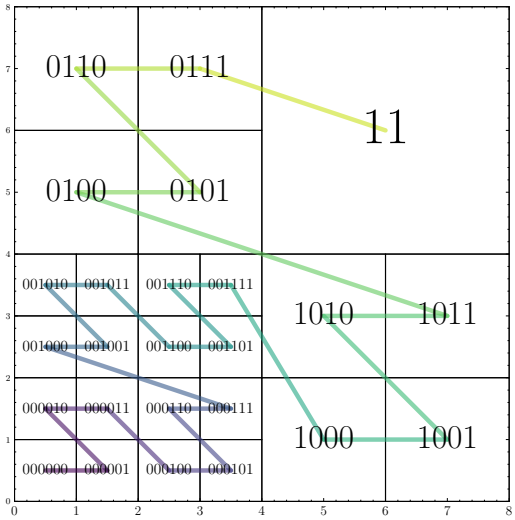
En la práctica, usaremos $L = 21$ para poder almacenar cada uno de los códigos en un entero sin signo de 64 bits, cuyo primer bit ignoraremos. Las coordenadas discretizadas $(\mathtt{x}, \mathtt{y}, \mathtt{z})$ se almacenarán temporalmente en enteros de 32 bits, ignorando los primeros 11 bits. Con esta precisión, es altamente improbable que más de un punto tenga asignada la misma celda, aunque esto podría ocurrir sin causar inconvenientes graves en el procesamiento de la nube, ya que su número será irrelevante en la práctica totalidad de aplicaciones. Así, el método para asignar los códigos a las celdas se basa en curvas de llenado del espacio, que se definen a continuación.

Definición 3.2. Una *curva de llenado del espacio (SFC)* de nivel l es una aplicación $C : S_l \to [0, 2^{3l}) \cap \mathbb{N}$. Decimos que una SFC es *recursiva* si $l = 1$ o si $l > 1$ y podemos dividir C en ocho SFCs $C_0, \ldots C_7$ recursivas de nivel $l - 1$ tales que exista una permutación $\pi : \{0, \ldots, 7\} \to \{0, \ldots, 7\}$ para la que $C_{\pi(i)}$ contiene los códigos $\{i2^{3(l-1)}, \ldots, (i+1)2^{3(l-1)} - 1\}$, para todo $i = 0, \ldots, 7$. Decimos que una SFC es *continua* si podemos conectar ortogonal o diagonalmente dos puntos consecutivos de su imagen[2], esto es:

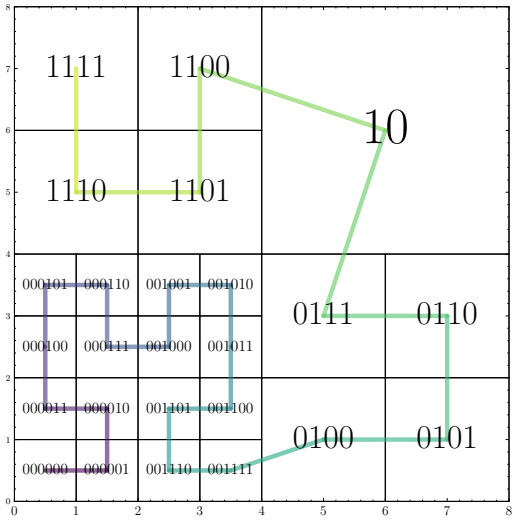
$$\left\| C^{-1}(i) - C^{-1}(i-1) \right\|_\infty = 1, \quad \text{para } i = 1, \ldots 2^{3l} - 1.$$

[2]Esta propiedad es equivalente a decir que $\mathcal{P}$ es continua en el espacio normado $(S_l, \|\cdot\|_\infty)$.

Es inmediato generalizar esta definición a un espacio k-dimensional, así como la de S_l y la discretización de la nube. En las figuras 3.1a y 3.1b se puede apreciar cómo las SFC recursivas de Morton y Hilbert, que describiremos a continuación, subdividen el espacio bidimensional. Este proceso, realizado en 3 dimensiones, subdivide el espacio en 8 octantes y será la base de la construcción de las estructuras *Octree* que usaremos más adelante.



(a) Proceso de subdivisión del espacio bidimensional mediante la SFC recursiva de Morton



(b) Proceso de subdivisión del espacio bidimensional mediante la SFC recursiva de Hilbert

Figura 3.1: SFCs bidimensionales.

3.2.1. Curva de Morton

Llamamos curva Z, de Lebesgue o de Morton [26] a la familia de SFCs recursivas obtenidas mediante el *entrelazamiento de los bits* de cada punto discretizado. Para un $p = (\mathtt{x}, \mathtt{y}, \mathtt{z}) = (\mathtt{x}_0 \dots \mathtt{x}_{L-1}, \mathtt{y}_0 \dots \mathtt{y}_{L-1}, \mathtt{z}_0 \dots \mathtt{z}_{L-1}) \in S_L$, tenemos:

$$C_M(\mathtt{x}_0 \dots \mathtt{x}_{L-1}, \mathtt{y}_0 \dots \mathtt{y}_{L-1}, \mathtt{z}_0 \dots \mathtt{z}_{L-1}) = \mathtt{x}_0\mathtt{y}_0\mathtt{z}_0 \dots \mathtt{x}_{L-1}\mathtt{y}_{L-1}\mathtt{z}_{L-1}.$$

Por ejemplo, tomemos $L = 3$, y un punto $\mathtt{p} = (3, 7, 2) = (011, 111, 010) \in S_3$. Su código de Morton será $C_M(\mathtt{p}) = 010111110 \in [0, 2^9)$. La decodificación sigue el proceso inverso, extrayendo los bits entrelazados a los 3 enteros originales. Nótese que hay 8 posibilidades para el orden en que entrelazamos cada uno de los bits, por eso hablamos de *familia* de SFCs. Cada uno de estas posibilidades se corresponde con el orden de los giros que realiza la curva. Para establecer nuestra

propuesta, no es relevante qué orden de entrelazamiento se escoja. En la figura 3.2 se muestra la curva en tres dimensiones, para los tres primeros niveles de profundidad.

Una propiedad conveniente de los códigos de Morton es la rapidez de su cálculo. Mediante el uso de *bitmasks* y *look-up tables* (LUTs), es posible obtenerlos con tan solo unas pocas instrucciones. La librería *libmorton* [3] proporciona implementaciones optimizadas en C++ para la codificación y decodificación para los casos $L = 21$ (64 bits) y $L = 10$ (32 bits), empleando además operaciones vectorizadas mediante BMI2 y AVX512.

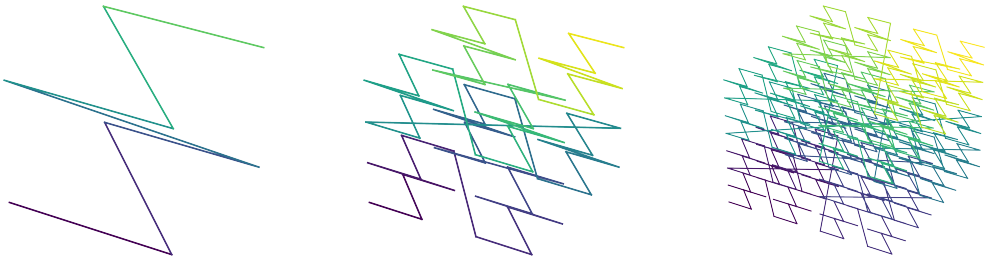


Figura 3.2: Curva de Morton tridimensional sobre S_1, S_2 y S_3.

3.2.2. Curva de Hilbert

La curva de Hilbert es otra familia de SFCs recursiva obtenida mediante un proceso iterativo en el que primero se extran los bits de cada nivel $\mathtt{x}_l\mathtt{y}_l\mathtt{z}_l$, al igual que en la curva de Morton. A continuación, se realizan una serie de rotaciones, dependientes del estado de giro previo y los bits extraídos en el nivel actual [24]. Estas rotaciones pueden ser descritas mediante un tipo de gramáticas de contexto libre (CFG) denominadas *sistemas de Lindermayer* [2, 6].

Una ventaja de la curva de Hilbert frente a la de Morton es que, gracias a los giros realizados, esta es continua. Como observamos en la figura 3.1b, no existen saltos de más de una celda entre dos códigos consecutivos, al contrario que en la figura 3.1a, donde se producen al pasar de un sub-bloque a otro. Esta propiedad se mantiene en el caso tridimensional, que se muestra en la figura 3.3. La familia de las curvas de Hilbert tridimensionales es muy grande [18], ya que estas rotaciones pueden llevarse a cabo de muchas formas. Sin embargo, para nuestros propósitos, tan solo es importante la continuidad, común a todas las curvas de Hilbert, y que previsiblemente otorgará una localidad espacial mejorada frente a los códigos de Morton. Estos códigos son más costosos computacionalmente que los de Morton,

al tener que iterar por cada nivel de subdivisión, y no ser posible usar LUTs para su cálculo directo. En nuestra implementación usamos el algoritmo de codificación y decodificación descrito en [25, Apéndice A].

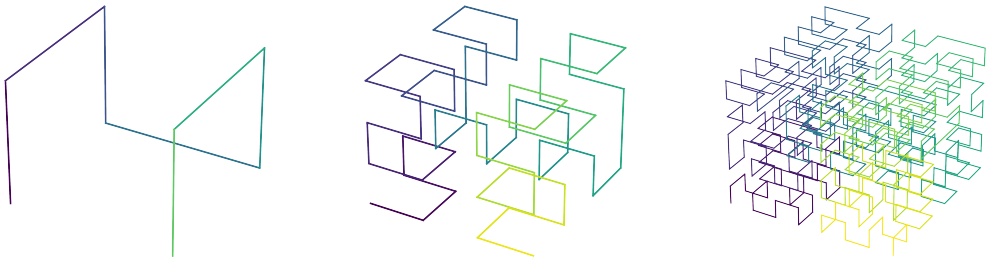


Figura 3.3: Curva de Hilbert tridimensional sobre S_1, S_2 y S_3.

3.2.3. Proceso de reordenamiento de la nube

A partir de las curvas de llenado del espacio, se puede finalmente reordenar la nube de puntos. Para ello, seguimos los siguientes pasos:

1. Para cada punto discretizado $\mathtt{p} \in S_L$, calculamos su codificación mediante una SFC de nivel L, obteniendo un array de códigos paralelo a la nube.

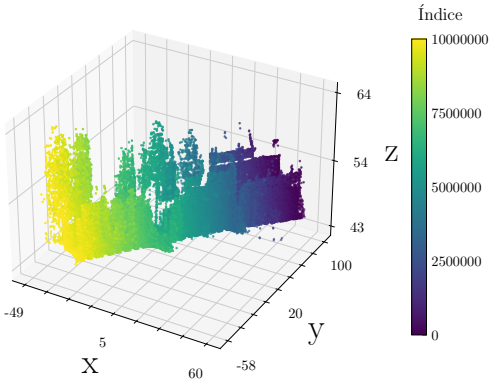
2. Reordenamos la nube $\mathcal{P}$ mediante el orden ascendente establecido por los códigos.

En cuanto al rendimiento de este proceso, el primer paso es trivialmente paralelizable, mientras que el paso segundo requiere un ordenamiento $\mathcal{O}(n \log n)$, que realizamos con el método `std::sort` de la librería estándar de C++.

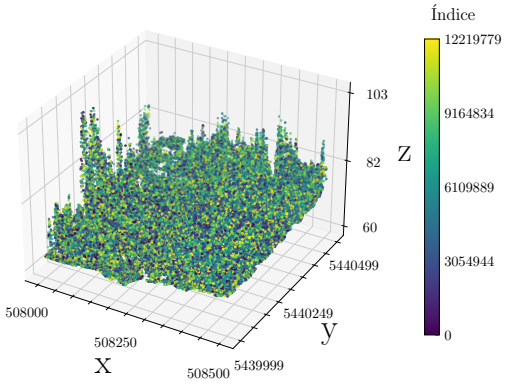
Aunque el reordenamiento es un paso relativamente costoso en nubes grandes, su coste computacional se puede amortizar realizándolo en un programa previo al principal. Este programa previo lee la nube de puntos desordenada y la escribe de nuevo con el orden especificado. En el formato *LAS* [30], podemos almacenar esta información adicional de reordenamiento en uno de los campos de datos de usuario presentes en el encabezado.

En la figura 3.4 se muestra cómo afecta el reordenamiento a las nubes de puntos *Lille_0*, del dataset *Paris_Lille* [31] y *5080_54400*, del dataset DALES [34]. Intuitivamente, el aumento en localidad se ve reflejado en la menor densidad

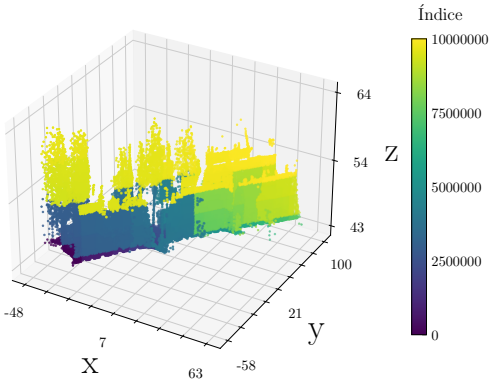
de rectas uniendo puntos consecutivos o en el orden más estructurado de los índices, de forma que la distancia promedio entre ellos es menor. Dependiendo del tipo de escáner LiDAR y de la topología del terreno capturado, los saltos entre puntos consecutivos en el orden original pueden ser mayores o menores, lo cuál se reflejará en un mayor o menor aumento en la localidad de la nube tras el reordenamiento. En este caso, la nube de DALES, captada con un sensor aéreo, presenta una menor localidad inicial.



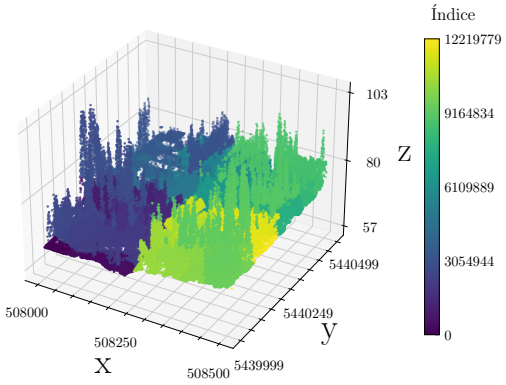
(a) Nube *Lille_0*, orden original.



(b) Nube *5080_54400* de DALES, orden original.



(c) Reordenamiento de *Lille_0* mediante la curva de Morton.



(d) Reordenamiento de *5080_54400* mediante la curva de Hilbert.

Figura 3.4: Reordenamientos de las nube de puntos *Lille_0* (10 millones de puntos, terrestre) y *5080_54400* (12 millones de puntos, aérea). La escala de color marca el orden en memoria en el que se encuentran los puntos. Por visibilidad, solo se muestran 100 000 puntos, uniformemente muestreados.

3.3. Estructuras Octree

Hasta este punto, la nube de puntos es tan solo un subconjunto de $\mathbb{R}^3$ no estructurado (aunque ya esté ordenado mediante una SFC). Bajo estas condiciones, tan solo podemos realizar búsquedas de vecinos con radio fijo o *k-NN* mediante métodos de fuerza bruta. Es habitual emplear estructuras con forma de árbol (*Bounding Volume Hierarchies*, BVH) como *kd*-trees [5], *R*-trees [16] u *Octrees* para particionar iterativamente el espacio en regiones cada vez más pequeñas y dar soporte a estas consultas espaciales. En este trabajo nos centraremos en los *Octrees*, que exhiben una gran regularidad y una baja profundidad, lo cual permite búsquedas de radio fijo muy eficientes. Además, existe una relación intrínseca entre la construcción de un *Octree* y el proceso de construcción de las SFC recursivas que ya hemos descrito.

La idea principal detrás de un *Octree* es la subdivisión recursiva de un volumen $O \subseteq \mathcal{B}$, llamado octante o nodo, en 8 *suboctantes* $O_1, \ldots, O_8$. Estos se corresponden con los 8 subvolúmenes obtenidos al cortar O por 3 planos perpendiculares a cada uno de los ejes cartesianos, cuya intersección es un punto dentro del volumen O.

La elección del punto de intersección define las proporciones entre las longitudes de los lados de cada octante. Un convenio habitual [4] es tomar p como el centro $c = (c_x, c_y, c_z)$ de O, de forma que los lados de los suboctantes sean del mismo tamaño $(x_{max} - x_{min})/2, (y_{max} - y_{min})/2, (z_{max} - z_{min})/2$. Esta elección simplificará los cálculos más adelante y refuerza la conexión con el proceso de subdivisión realizado en las SFC recursivas.

De forma general, el algoritmo 1 describe el proceso de construcción de un *Octree*. Nótese que en las implementaciones específicas que veremos más adelante el significado de crear un nodo e insertar puntos en él puede variar. Tampoco especificaremos, de momento, dónde se almacena cada uno de los nodos creados. La condición de división, *subdivide()*, será cierta si el número de puntos en el octante es mayor que un límite establecido N_{max} y la profundidad de la rama no es mayor que L.

Por lo tanto, las hojas del árbol contendrán no más de N_{max} puntos, salvo que se haya alcanzado la profundidad máxima. En la figura 3.5 se ilustran los bordes de los octantes de un *Octree* construido sobre la nube *Lille_0*, reordenada según la curva de Hilbert. Las áreas de mayor densidad de puntos se corresponden con una mayor profundidad.

Algoritmo 1 Construcción de un *Octree*

Entrada: $\mathcal{P}$: nube de puntos dentro del octante
(c_x, c_y, c_z): centro del octante
(r_x, r_y, r_z): radios del octante

1: **procedure** BUILDOCTREE($\mathcal{P}, c_x, c_y, c_z, r_x, r_y, r_z$)
2: createNode()
3: **if** SUBDIVIDE( ) **then**
4: **for** $i = 0, \ldots, 7$ **do**
5: *// Calcular los centros y radios de los suboctantes*
6: $R_x, R_y, R_z \leftarrow r_x/2, r_y/2, r_z/2$
7: $C_x \leftarrow c_x + ((i \,\&\, 4)\ ?\ R_x : -R_x)$
8: $C_y \leftarrow c_y + ((i \,\&\, 2)\ ?\ R_y : -R_y)$
9: $C_z \leftarrow c_z + ((i \,\&\, 1)\ ?\ R_z : -R_z)$
10: *// Dividir puntos entre suboctantes*
11: $\mathcal{P}_i \leftarrow \{p \in \mathcal{P} : p$ está en el suboctante $i\}$
12: *// Llamada recursiva*
13: BUILDOCTREE($\mathcal{P}_i, C_x, C_y, C_z, R_x, R_y, R_z$)
14: **end for**
15: **else**
16: **for all** $p \in \mathcal{P}$ **do**
17: *insertPoint*(p)
18: **end for**
19: **end if**
20: **end procedure**

3.3.1. Octrees basados en punteros

La forma más natural de implementar la estructura de *Octree* es hacerlo de forma similar a un árbol de búsqueda binaria, representando cada octante como un nodo con punteros a cada uno de sus hijos. En este caso, cada nodo tendrá 8 hijos, creados una vez se alcanza la condición de subdivisión y representando cada uno de los suboctantes.

En cada nodo del *Octree* se almacena su centro y su radio. En los nodos internos, se guarda un vector con los punteros a cada uno de sus suboctantes, mientras que en las hojas se guardará un vector con punteros a cada uno de los puntos contenidos en ella. También se puede almacenar el punto mínimo y máximo del suboctante, así como un puntero al nodo padre si es necesario recorrer hacia arriba la estructura. La implementación de la estructura usada en nuestras pruebas está descrita en detalle en [14].

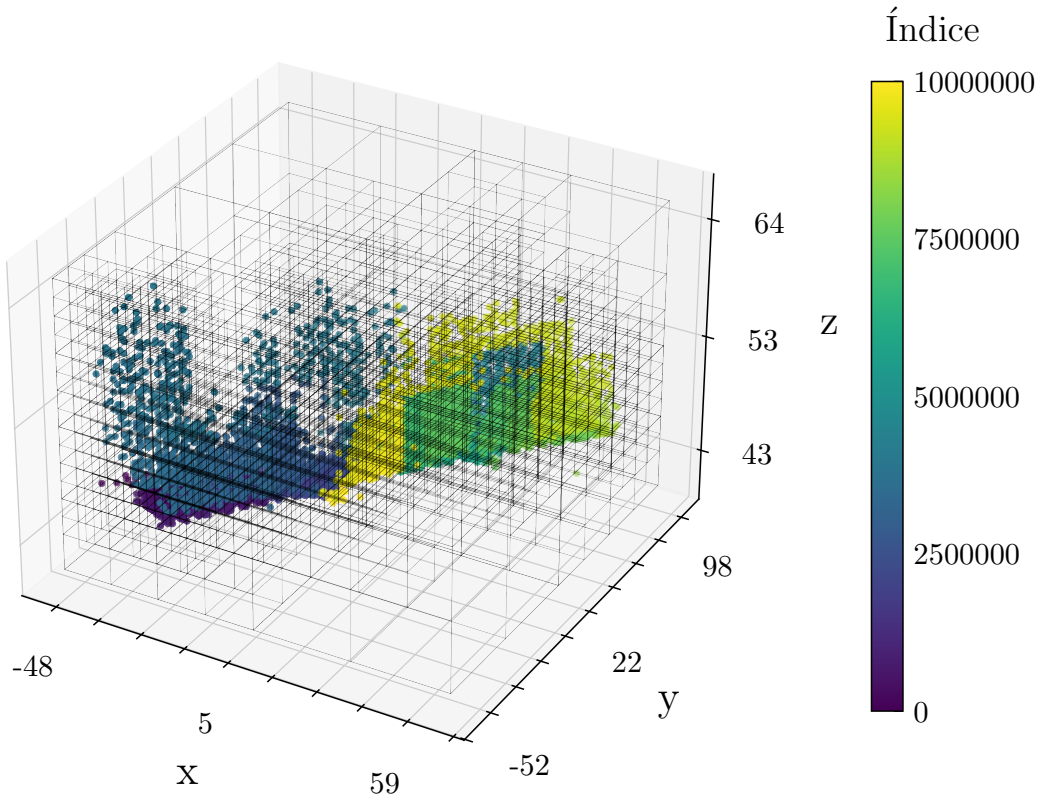


Figura 3.5: Bordes de los octantes de un *Octree* sobre la nube de puntos *Lille_0*, reordenada mediante la curva de Hilbert.

Para realizar búsquedas de vecinos con radio fijo, primero especificamos un centro, que suele ser un punto de la nube, un radio de búsqueda y un tipo de *kernel de búsqueda*, que definimos a continuación.

Definición 3.3. Un *kernel de búsqueda* es una aplicación $\mathcal{N} : (\mathcal{P}, \mathbb{R}^3, \mathbb{R}) \to 2^{\mathcal{P}}$ que obtiene la vecindad de un punto, siguiendo una distancia específica, a partir de la nube de puntos $\mathcal{P}$, un centro $c \in \mathbb{R}^3$ y un radio $r > 0$. A lo largo de este trabajo, usaremos los siguientes 4 kernels:

- **Esfera**: $\mathcal{N}_{Sphere}(\mathcal{P}, c, r) = \{p \in \mathcal{P} : \|(p_x, p_y, p_z) - (c_x, c_y, c_z)\|_2 < r\}$.
- **Círculo**: $\mathcal{N}_{Circle}(\mathcal{P}, c, r) = \{p \in \mathcal{P} : \|(p_x, p_y) - (c_x, c_y)\|_2 < r\}$.
- **Cubo**: $\mathcal{N}_{Cube}(\mathcal{P}, c, r) = \{p \in \mathcal{P} : \|(p_x, p_y, p_z) - (c_x, c_y, c_z)\|_\infty < r\}$.
- **Cuadrado**: $\mathcal{N}_{Square}(\mathcal{P}, c, r) = \{p \in \mathcal{P} : \|(p_x, p_y) - (c_x, c_y)\|_\infty < r\}$.

El tipo de kernel a utilizar depende de la aplicación específica. Los kernels bidimensionales (círculo y cuadrado) tienden a utilizarse sobre nubes planas co-

mo las obtenidas a partir de los métodos ALS o ULS. Como se observará más adelante, el rendimiento de los kernels en norma infinito es evidentemente más rápido, al sustituir multiplicaciones por comparaciones durante el cálculo de las normas.

Para calcular estas vecindades, se debe recorrer el árbol mediante una búsqueda en profundidad, buscando los octantes que contienen las hojas con los puntos dentro del *kernel*. Para ello, comenzando en la raíz del *Octree*, se realizan los siguientes pasos:

1. Se comprueba si el nodo actual no es una hoja. De ser así, calculamos cuáles de sus hijos se intersecan con el *kernel*.
2. Para los hijos que intersecan, repetimos el proceso hasta llegar a las hojas del árbol.
3. Para cada hoja que interseca, comprobamos punto a punto si pertenecen o no al *kernel* propuesto.
4. Finalmente, se devuelven todos los puntos en el interior del *kernel*.

Este método de búsqueda en profundidad implica recorrer cada uno de los niveles del *Octree* hasta llegar a las hojas o encontrar una rama completamente fuera del kernel. En nubes de puntos de alta densidad, los pasos de redirección de punteros necesarios se pueden volver costosos, ya que no existe forma directa de obtener, a partir de una rama, los puntos contenidos en ella. Además, lo que se guarda en las hojas del árbol son realmente direcciones de memoria a la localización de cada uno de los puntos (o, equivalentemente, sus índices en el vector que contiene la nube), con el objetivo de no duplicar todos los datos de la nube en el *Octree*. Posteriormente, veremos cómo se pueden subsanar estos problemas mediante la linearización del árbol.

3.3.2. Octrees lineales

Con el objetivo de simplificar la estructura y ligarla con la reordenación de la nube mediante SFCs recursivas, se han desarrollado múltiples variantes de la estructura *Octree* en las que no se usan punteros para recorrer el árbol, que llamaremos *octrees lineales o linearizados*. Una de las variantes principales son los *Hashed Octrees* [36, 9], en las que cada nodo se guarda en una tabla hash. Para construir el índice de esta tabla, se utiliza un formato similar al de los códigos de Morton, llamado formato de Warren-Salmon [36], en el que también se codifica el nivel del árbol correspondiente de la siguiente forma:

- **Caso base:** El nodo raíz, conteniendo toda la bounding box $\mathcal{B}$, tendrá el código 1, llamado *placeholder bit*.

- **Caso recursivo:** Cada uno de los suboctantes O_i, $i = 0, \ldots, 7$ de un octante O con código key tendrá el código $key \cdot 8 + i$. Por ejemplo, los nodos en el segundo nivel del árbol tendrán los códigos $1000, 1001, 1010, \ldots, 1111$, manteniendo el *placeholder bit* que marca el nivel actual.

Esta implementación da acceso directo a cada nodo, ya sea una rama o una hoja, sin importar su profundidad, que se puede determinar a partir del número de ceros antes del primer uno. Esto supone un primer paso hacia la resolución de los problemas del *Octree* basado en punteros. Sin embargo, con los *Hashed Octrees* seguimos sin poder acceder a todos los puntos en las hojas contenidas en una rama directamente. Además, sigue existiendo otro de los problemas: la fragmentación de memoria de los nodos. Tanto en el proceso de subdivisión del *Octree* basado en punteros como en la inserción de entradas en una tabla hash no existen garantías de que los nodos del árbol se mantengan en posiciones de memoria contiguas, lo cual limita la localidad durante las búsquedas de vecinos.

Cornerstone Octree

Para abordar los problemas expuestos anteriormente, adaptaremos la estructura *cornerstone-octree*, desarrollada por Keller et al. [23]. Se trata de un *Octree* lineal combinando dos representaciones distintas, una exclusivamente para las hojas y otra que también incluye los nodos internos del *Octree*. En esta estructura, en lugar de almacenar el árbol explicitamente o usar mapas para relacionar los códigos de cada nodo con sus datos, sintetizamos toda la información necesaria en una serie de arrays. La representación de las hojas del *Octree* sigue el formato *cornerstone-array*, que se describe a continuación. Por otro lado, los nodos internos seguirán el formato de Warren-Salmon, al igual que en los *Hashed-Octrees*, pero sin necesidad de usar tablas hash.

El formato *cornerstone-array* se aplica en un array $leaves = \{K_0, \ldots, K_{n_l}\}$, de tamaño $n_l + 1$, cuyos elementos identifican a cada hoja del *Octree*. Estos elementos no son más que números binarios representando una región del espacio, de forma que todos los puntos de la nube en la hoja i-ésima tendrán un código SFC contenido en el intervalo $[K_i, K_{i+1})$. Estas hojas se pueden subdividir, de forma similar al cálculo iterativo de los códigos de Morton o de Hilbert. Definimos

cuatro invariantes sobre *leaves*:

$$\begin{aligned} K_0 &= 0, \\ K_{n_l} &= 8^L, \\ K_i &< K_j, \quad 0 \leq i < j \leq n_l, \\ K_{i+1} - K_i &= 8^l, \quad 0 \leq i < n_l, \ 0 \leq l \leq L \end{aligned}$$

Para construir *leaves*, seguimos los siguientes pasos:

1. Se consideran los puntos de la nube ordenados según una SFC recursiva (de Morton o de Hilbert). También se necesitan sus códigos ordenados, presentes en un array *codes*. Partimos de $leaves = \{0, 8^L\}$.

2. Se determina cuántos puntos están en cada uno de los rangos $[K_i, K_{i+1})$, es decir, cuántos puntos están en cada hoja. Para esto, utilizamos búsqueda binaria sobre los códigos.

3. Si hay más puntos en $[K_i, K_{i+1})$ que el umbral N_{max}, entonces se decide subdividir este rango en 8 partes iguales. Por ejemplo, para la primera subdivisión, haremos $\{0, 8^L\} \to \{0, 8^{L-1}, \ldots, 7 \cdot 8^{L-1}, 8^L\}$. Una vez hemos comprobado, para cada elemento de *leaves*, si hace falta subdividirlo, construimos el nuevo array *leaves* y reemplazamos el antiguo. Si ha habido cambios en *leaves*, volvemos al paso anterior.

La implementación del paso 2 es la clave de este algoritmo. Para ello, utilizaremos el array de códigos SFC de la nube *codes*, que, antes de comenzar la construcción del *Octree*, ya ha sido ordenado de forma ascendente junto al array que contiene la nube $\mathcal{P}$. La propiedad fundamental que conserva cada hoja es la siguiente: si tenemos un punto $p_j \in \mathcal{P}$, cuyo código SFC es c_j, entonces la hoja i del árbol contiene al punto p si y solo si $K_i \leq c_j < K_{i+1}$. Por lo tanto, podemos emplear búsqueda binaria para encontrar el rango de índices de puntos $[l_i, u_i)$ que contiene cada hoja:

$$\begin{aligned} l_i &= \text{máx}\left\{j \in \{0, 1, \ldots, N\} : c_j \leq K_i\right\}, \\ u_i &= \text{mín}\left\{j \in \{0, 1, \ldots, N\} : c_j > K_i\right\}. \end{aligned}$$

Una vez obtenidos estos rangos, es inmediato realizar el paso 3, calculando el número de puntos en cada hoja $u_i - l_i$ y dividiendola cuando este sea mayor que el umbral N_{max}. Los tamaños de cada hoja se almacenan en un array *counts*, es decir, $counts[i] = u_i - d_i$. El *scan exclusivo* de *counts*, que llamaremos *layout*, proporcionará el primer índice de la nube $\mathcal{P}$ donde se encuentran los puntos correspondientes a una hoja i.

Ejemplo 3.1 (Construcción de *leaves*)**.** Consideremos la nube de puntos $\mathcal{P}$, de tamaño $N = 7$. Sea $L = 3$ el nivel máximo de subdivisión y $N_{max} = 2$ el número máximo de puntos por hoja. Por simplicidad, este ejemplo se hará en el espacio bidimensional (construyendo un *Quadtree* lineal en vez de un *Octree*). Las coordenadas discretizadas de los puntos de la nube (ya ordenada) serán $\{(0,0),(1,0),(1,3),(3,1),(2,5),(6,1),(6,6)\}$ y sus códigos de Morton (en base 4) serán $\{000, 002, 013, 023, 121, 221, 330\}$. El *Octree* lineal se puede obtener del siguiente modo:

Paso 1: *leaves* $= \{0, 1000\}$, *counts* $= \{7, 0\}$ y *layout* $= \{0, 7\}$. Como la primera hoja tiene más de 2 puntos, debemos subdividirla en 4.

Paso 2: *leaves* $= \{0, 100, 200, 300, 1000\}$, *counts* $= \{4, 1, 1, 1, 0\}$, *layout* $= \{0, 4, 5, 6, 7\}$. Al igual que en el paso previo, la primera hoja tiene demasiados puntos, la subdividimos de nuevo.

Paso 3: *leaves* $= \{0, 010, 020, 030, 100, 200, 300, 1000\}$, *counts* $= \{2, 1, 1, 0, 1, 1, 1, 0\}$, *layout* $= \{0, 2, 3, 3, 4, 5, 6, 7\}$. Como ningún elemento de *counts* es mayor que N_{max}, no hacen falta más subdivisiones.

En la figura 3.6 se muestra la nube a lo largo de cada paso de subdivisión.

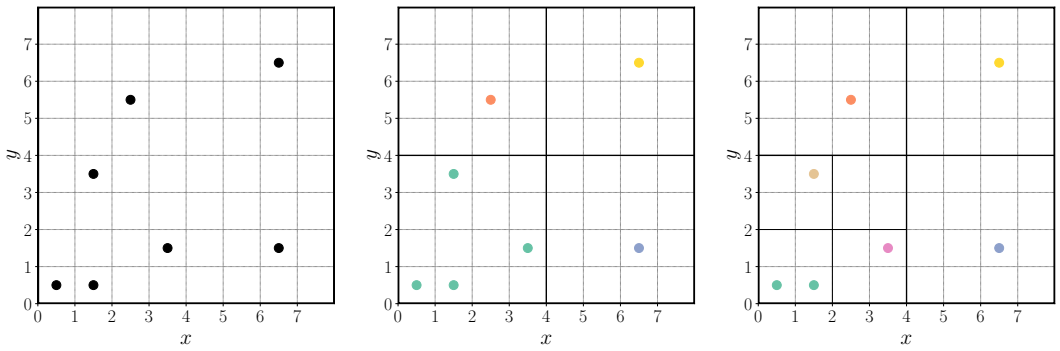


Figura 3.6: Visualización de la nube a través de cada paso del proceso de subdivisión en cuadrantes.

Una vez hecho esto, es necesario realizar un segundo paso en el que se calcula la *parte interna* del árbol, necesaria para realizar los recorridos en profundidad en los que se basan nuestros algoritmos de búsqueda de vecinos. Para ello, enlazaremos el árbol mediante el cálculo de otros dos arrays, *prefixes* y *offsets*:

1. *prefixes* contiene códigos para los nodos internos y las hojas del árbol en el formato de Warren-Salmon.

2. *offsets* contiene, para los nodos internos, el índice del primer hijo de este nodo. Para las hojas, su valor es 0.

Como en *leaves* siempre subdivimos las hojas en 8, es sencillo demostrar que si tenemos n_l hojas, entonces habrá $n_i = \frac{n_l - 1}{7}$ nodos internos. Por ejemplo, dado un *Octree* completo de 2 niveles, entonces tendremos $n_l = 64$ hojas y $n_i = 9$ nodos internos, correspondiéndose con la raíz, los 8 nodos de nivel 1 y las 64 hojas de nivel 2. Los arrays *prefixes* y *offsets* serán de tamaño $n = n_l + n_i$.

Ejemplo 3.2 (Parte interna del *Octree* lineal)**.** Continuemos con el ejemplo 3.1, como $n_l = 7$ entonces habrá $n_i = 2$ nodos internos, correspondiéndose con la raíz y el primer cuadrante de tamaño 4×4 en el nivel 1. En la figura 3.7 se ilustra el árbol obtenido tras el proceso de construcción de la parte interna, donde los números representan el código de *prefixes* en formato de Warren-Salmon. Los subíndices representarán la posición del nodo interno (es decir, la posición en el array *prefixes*), que acabará estando en orden de búsqueda en profundidad y los superíndices el valor de *offsets* en esta posición interna. Para las hojas, también se añade una segunda línea con el código en formato *cornerstone-array* del elemento correspondiente en el array *leaves*, así como un subíndice con su posición en *leaves*.

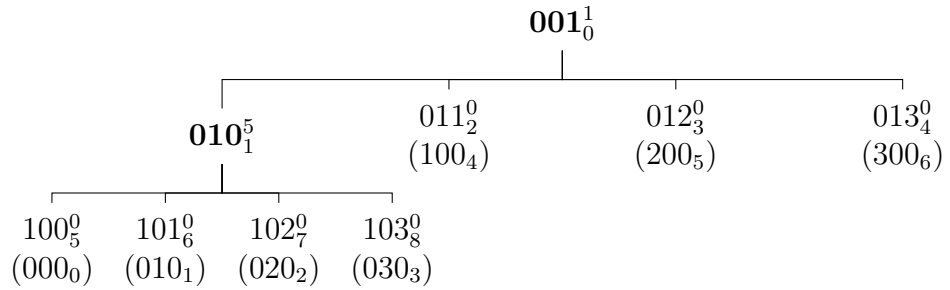


Figura 3.7: Parte interna del *Octree* lineal construido sobre la nube del Ejemplo 3.1.

Para generar *prefixes*, primero se deben convertir las hojas de *leaves* al formato de Warren-Salmon. Para ello, basta hallar su nivel de subdivisión contando ceros desde la derecha del código de la hoja, desplazar los bits y activar el bit marcando el nivel de subdivisión. A continuación, estos códigos se copian al subarray $prefixes[n_i], \ldots, prefixes[n-1]$.

Para calcular los códigos de los nodos internos de forma eficiente, seguimos el proceso indicado por Keller et al. [23], basado en los algoritmos paralelos de Karras [22] y Apetrei [1] para la construcción de estructuras de volúmenes lineales.

Para cada hoja K_i con $i = 0, 1, \ldots, n_l - 2$, tomamos $c = clz(K_i \oplus K_{i+1})$, donde clz es el número de ceros antes del primer uno y $\oplus$ el operador XOR. Si se cumple $c \equiv 0 \pmod 3$, entonces existe un nodo interno de nivel $c/3$ entre ambas hojas. El índice j de este nodo interno en *prefixes* viene dado por la ecuación (3.2)

$$j = \frac{i - \delta(K_i, c)}{7}. \tag{3.2}$$

Donde la función $\delta(K_i, c)$ expresa cierto desplazamiento entre los índices de *leaves* y los de *prefixes*, definida, para una hoja en base octal $K_i = (d_0 d_1 \ldots d_L)_8$ según la ecuación (3.3)

$$\delta(K_i, c) = \sum_{l=0}^{c/3+1} \mu_{d_l}, \quad \text{donde } \mu = (0, -1, -2, -3, 3, 2, 1, 0). \tag{3.3}$$

Finalmente, se reordena el vector *prefixes* ascendentemente. Gracias a las propiedades de los códigos de Warren-Salmon, este reordenamiento se corresponde con un orden de búsqueda en anchura (BFS) sobre el *Octree*, donde los 8 hijos de cada nodo se encontrarán en posiciones consecutivas. Para localizar exactamente donde se encuentran y acabar el enlazamiento del árbol, debemos calcular el array *offsets*. Este proceso se puede realizar mediante búsqueda binaria: para un nodo interno con código $offsets[i] = key$, buscamos en el array ordenado *prefixes* el índice j con el código de su primer hijo, $prefixes[j] = 8 \cdot key$. De este modo, $offsets[i] = j$.

Aprovechándonos del ordenamiento BFS de *prefixes*, el cálculo de *offsets* se puede optimizar en *Octrees* grandes precomputando un array *levelRange*, de tamaño $L + 2$. Este array marca el último índice de un nivel dado $l = 1, \ldots, L$, siendo además $levelRange[0] = 0$ y $levelRange[L + 1] = n$. Por ejemplo, si tenemos un árbol completo de 2 niveles con 64 hojas y 9 nodos internos, obtenemos $levelRange = (0, 1, 9, 73, 73, \ldots, 73)$. Este array se puede obtener con tan solo L búsquedas binarias de los elementos 8^l en *prefixes*. Una vez calculado, y obteniendo el nivel l en el que se encuentra un nodo interno, podemos reducir el rango de búsqueda binaria a los índices $(levelRange[l + 1], levelRange[l + 2])$ de *prefixes*, ya que el primer hijo de este nodo se encontrará en el nivel $l + 1$.

Con esto, finalizamos la descripción de la construcción de la parte interna del árbol. Cabe destacar que, gracias a la organización del proceso de subdivisión y enlazado en varias etapas, cada paso es altamente paralelizable, tanto en arquitecturas multihilo como en GPUs. Los pasos con dependencias de datos no triviales se pueden descomponer en algoritmos como scans paralelos o reordenamientos mediante *radix sort*.

Paralelizar la construcción del árbol es importante en aplicaciones que necesitan reconstruir el *Octree* a menudo, o cuando tratamos con grandes nubes de miles de millones de puntos, como el contexto de simulaciones de partículas de Keller et al. [23]. En nuestro caso, solo hemos paralelizado los bucles sin dependencias, pero no hemos implementando los algoritmos de *parallel scan* ni el reordenamiento de *prefixes* mediante *radix sort*, que se pueden usar para optimizar aún más el proceso. En cualquier caso, como las nubes LiDAR son consideradas estáticas, el *Octree* no necesita reconstruirse, por lo que la construcción rápida de este no es tan importante.

Una vez completada la construcción del *cornerstone-octree*, es posible realizar un recorrido preliminar en el que expandimos el array *ranges* de las hojas a los nodos internos. Mediante una búsqueda en profundidad, se puede calcular el array de pares *internalRanges*, expresando el rangos de índices de la nube contenido en cualquier nodo, hoja o interno, del *Octree* lineal. Esta es nuestra principal adaptación a la estructura y nos proporciona acceso directo a la lista de puntos contenidos en cada rama del árbol, lo cuál será clave para acelerar aún más la búsqueda de vecinos. Finalmente, obtenemos una estructura con las siguientes ventajas sobre el *Octree* basado en punteros:

- Se eliminan los pasos de redirección de punteros que pueden provocar fallos caché durante búsquedas en árboles profundos, así como la fragmentación en memoria del árbol. La estructura queda completamente contenida en unos pocos bloques de memoria.
- Se conserva acceso directo a la nube de puntos en cualquier nodo del árbol, sea interno o una hoja, y se conoce el rango de puntos presente en él. Además, gracias a la reordenación por SFCs, los puntos de cada rango se encuentran en posiciones consecutivas en memoria.

En el cuadro 3.2 se recopilan los arrays obtenidos. Cabe destacar que tan solo *internalRanges* y *prefixes* son necesarios para las búsquedas de vecinos. Todos los demás son intermedios y se pueden eliminar tras la construcción del árbol para ahorrar espacio. En la estructura final, también precomputamos el array auxiliar *centers*, conteniendo el centro de cada octante. Para los radios de los octantes, basta guardar uno por nivel y llevar la cuenta del nivel de profundidad actual en las búsquedas. Esto mejora ligeramente el rendimiento de las búsquedas, ya que estos cálculos conllevan divisiones en punto flotante.

El punto débil de esta estructura es el coste de añadir o borrar nuevos puntos. Este proceso, relativamente sencillo en *Octrees* basados en punteros, es más delicado en el *cornerstone-octree*, ya que puede implicar reconstruir toda la estructura en el caso de que sea necesaria una división de octante. Como ya hemos

Cuadro 3.2: Resumen de cada uno de los componentes de un *cornerstone-octree*.

Nombre	Parte	Descripción
leaves	Hojas	Hojas del árbol en formato *cornerstone-array*
counts	Hojas	Número de puntos de la nube en cada hoja
layout	Hojas	Primer índice en $\mathcal{P}$ de los puntos de una hoja
prefixes	Interna	Códigos de cada nodo interno (hoja o rama) en el formato de Warren-Salmon
offsets	Interna	Índices en *prefixes* del primer hijo de cada nodo
internalRanges	Interna	Rangos de índices de puntos bajo cada nodo
centers	Interna	Centros de los nodos

comentado, esto se puede remediar con la paralelización completa del proceso de construcción, pero no consideraremos este inconveniente, al asumir que las nubes de puntos son estáticas. En la siguiente sección, describiremos los algoritmos de búsqueda desarrollados sobre este *Octree* lineal.

3.4. Algoritmos de búsqueda de vecinos optimizados

Aunque las búsquedas de vecinos se pueden realizar de la misma forma que en el *Octree* basado en punteros, la nueva información permite optimizar este proceso significativamente.

3.4.1. Inserción directa de puntos con octante contenido en kernel

La primera optimización consiste en la poda del árbol durante las búsquedas en profundidad cuando el octante actual está completamente contenido en el kernel de búsqueda, de forma que, si se continuara descendiendo, se insertarían todos los puntos. Para poder hacer esto de forma eficiente, refinamos la comprobación geométrica que decide si un octante interseca o está fuera del kernel de búsqueda $\mathcal{N}(p, c, r)$ con un caso adicional indicando si el octante está completamente dentro del kernel.

Gracias al alineamiento con los ejes, este cálculo es muy rápido. Para los kernels con norma $\|\cdot\|_\infty$, cubo y cuadrado, basta realizar una serie de desigualdades entre cada extremo, al ser el octante en cuestión también un cubo. Para los ker-

nels con norma $\|\cdot\|_2$, esfera y círculo, se puede comprobar si cada una de las 8 esquinas del octante está contenida en el kernel. Sin embargo, es posible optimizar este proceso empleando las simetrías del octante. Utilizamos la implementación rápida dada por [4], en la cual, mediante un análisis de casos y una transformación al sistema de coordenadas local del octante, se reduce el número de cálculos de norma a tan solo uno.

Una vez hemos encontrado un octante contenido en el kernel y podamos la búsqueda, es necesario insertar todos los puntos bajo ese octante en el vecindario. Para ello, utilizamos el array *internalRanges*, el cuál contiene el rango de puntos bajo el octante a insertar. Gracias a la reordenación SFC, estos puntos son además consecutivos en memoria, lo cuál acelera su copia al resultado. El pseudocódigo para esta implementación, que llamaremos *neighborsPrune* en nuestras pruebas, se muestra en el algoritmo 2.

Algoritmo 2 Búsqueda de vecindades optimizada *neighborsPrune*

Entrada: $\mathcal{P}$: nube de puntos
k, r, c: tipo de *kernel* (esfera, cubo, ...), radio y centro
oct: octante actual (índice de nodo interno), originalmente la raíz
Salida: v: vector de vecindades

```
1:  procedure NEIGHBORSPRUNE(P, k, r, c, oct)
2:      if oct es una hoja then
3:          v ← v ∪ {p ∈ oct : p está dentro del kernel de búsqueda}
4:      else
5:          if oct está completamente dentro del kernel then
6:              // Insertamos todos los puntos
7:              v ← v ∪ {p ∈ P : p está en una hoja por debajo de oct}
8:          else if oct interseca con el kernel then
9:              for i = 1, ..., 8 do
10:                 // oct_i es el suboctante i-ésimo de oct.
11:                 v ← v ∪ NEIGHBORSPRUNE(P, k, r, c, oct_i)
12:             end for
13:         end if
14:     end if
15:     return v
16: end procedure
```

Este método es más rápido por varias razones. En primer lugar, se visitan menos nodos y comprueban menos hojas y puntos, lo cuál ahorra un gran número de comprobaciones geométricas. Además, la inserción de puntos en el resultado de forma secuencial durante la poda de nodos internos es mucho más rápida.

3.4.2. Estructura basada en rangos para el resultado

La siguiente optimización es un pequeño cambio en la implementación del anterior método, que, no obstante, representa una gran mejora en la eficiencia y escalabilidad de las búsquedas. Hasta este momento, el resultado de las búsquedas ha sido un array de punteros a cada uno de los puntos de la nube $\mathcal{P}$ dentro del *kernel* de búsqueda. Aunque esto consume menos memoria y es ligeralmente más rápido que copiar la información de cada punto en el resultado, insertar cada una de estas direcciones de memoria es costoso.

Además, el número de puntos a insertar crece rápidamente al aumentar el radio (para una nube $\mathcal{P}$ completamente uniforme, sería $\mathcal{O}(r^3)$ en los *kernels* esfera y cubo, y $\mathcal{O}(r^2)$ para los *kernels* círculo y cuadrado) por lo que estas inserciones pueden hacerse demasiado costosas. Otra forma similar de representar el vecindario es insertar los índices en la nube de cada punto, sin embargo, utilizando índices de 8 bytes, esto no mejora el rendimiento.

Proponemos evitar muchas de estas inserciones pasando a almacenar un array de rangos *neighRanges*, con $neighRanges[i] = [l_i, u_i)$, conteniendo todos los índices de los puntos de la nube $\mathcal{P}$ presentes en el *kernel* de búsqueda. Para ello, tan solo se debe modificar la inserción de puntos iterativa durante la poda, de forma que se inserte el rango $internalRanges[j]$ en *neighRanges*, donde j es el índice interno del octante a podar. Además, también se pueden crear estos rangos al encontrar varios puntos consecutivos en las hojas dentro de un mismo kernel, comprimiendo aún más el resultado final.

La observación principal detrás de esta idea es que, al aumentar el radio de búsqueda y encontrar miles de puntos por búsqueda, el tamaño del resultado se convierte en una limitación importante para la velocidad, igual de relevante que la localidad de la nube de puntos o el número de comprobaciones geométricas realizadas. Para facilitar el uso posterior del vecindario obtenido, se encapsuló este array de rangos en una estructura simple con un iterador de C++ y un indicador del número total de puntos presentes en el vecindario. Cabe destacar que la iteración sobre esta estructura no es más lenta que sobre un vector de índices o punteros, por lo cuál el tiempo de post-procesamiento de los resultados no se debería ver afectado.

En nuestras pruebas, llamaremos *neighborsStruct* a este método. Como la probabilidad de que un octante i en el nivel $l \leq L$ esté completamente contenido en el *kernel* aumenta con su radio r, este método de búsqueda escala mejor que el anterior para grandes radios de búsqueda.

3.4.3. Búsquedas de vecinos aproximadas

Finalmente, proponemos un método de aproximación de los *kernels* en el que finalizamos las búsquedas antes de llegar a las hojas, llegando a insertar o ignorar todo el rango de puntos bajo una rama a partir de cierto nivel. Este método, que llamaremos *neighborsApprox*, necesita dos argumentos: un booleano *upperBound* y una tolerancia *tol*. La tolerancia es usada para el cálculo del nivel máximo del *Octree* al cual llegaremos, dejando de descender cuando todos los radios del octante actual sean menores que $r \cdot tol$. Como todos los octantes en un mismo nivel l tienen el radio $r_l = \big((x_{max} - x_{min})/2^l, (y_{max} - y_{min})/2^l, (z_{max} - z_{min})/2^l\big)$, es inmediato encontrar el nivel máximo de la búsqueda l_{max} antes de comenzar el recorrido sobre el árbol. Cuando *upperBound* es cierto, realizamos una búsqueda de cota superior del kernel, insertando todos los puntos en cada octante encontrado en el nivel l_{max}. En el caso contrario, realizamos una búsqueda de cota inferior, ignorando los puntos de los octantes encontrados en el nivel l_{max}. El resto del algoritmo es idéntico a *neighborsStruct*. Por supuesto, cuanto más se aumenta el parámetro de tolerancia, mejor se aprovecha la estructura de rangos, que permite añadir grandes cantidades de puntos al resultado rápidamente. En la figura 3.8 podemos ver como la tolerancia afecta el vecindario obtenido.

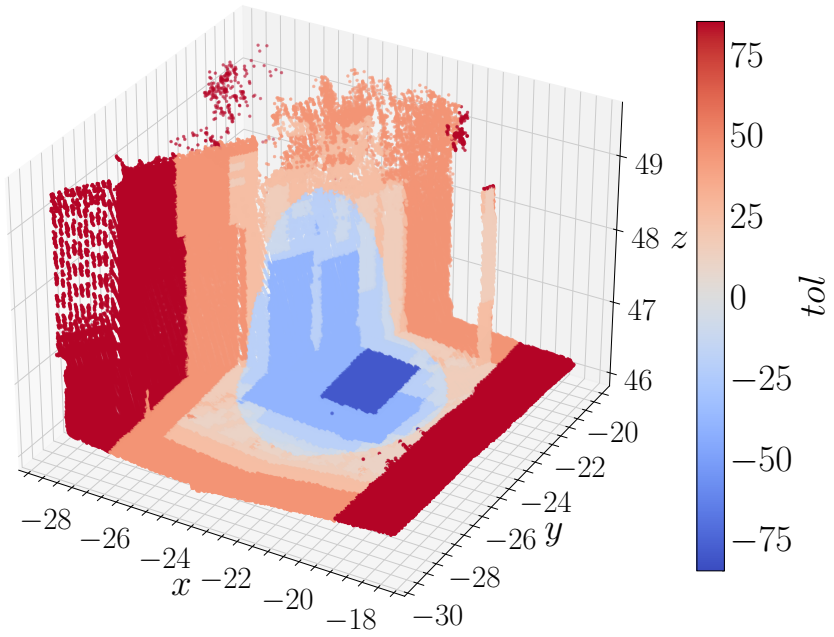


Figura 3.8: Búsqueda de vecinos aproximada sobre un centro de la nube *Lille_0*, con *kernel* esférico de radio 3 y varias tolerancias. Los valores negativos de tolerancia se corresponden con búsquedas de cota inferior.

Evidentemente, esta aproximación conlleva una pérdida de precisión. En nuestras pruebas, nos limitamos a medir las diferencias en el tamaño del resultado, siendo necesario un análisis más detallado para aplicaciones y nubes específicas, así como una elección correcta de la tolerancia y el modo de cota superior o inferior. Además, aunque la estructura basada en rangos enmascara los tiempos de inserción de puntos en el resultado, la iteración sobre cada uno de los puntos sigue llevando un tiempo similar. Esto es relevante para las búsquedas aproximadas de cota superior, pero pudiendo crear un vecindario demasiado grande, costoso de iterar.

Capítulo 4

Pruebas

A lo largo de este capítulo, pondremos a prueba los algoritmos de búsqueda y las estructuras *Octree* descritas. Para ello, realizaremos una serie de búsquedas paralelas de vecinos sobre un conjunto de centros de la nube, unos radios y unos tipos de kernel específicos para cada caso. Nuestro objetivo es demostrar que las optimizaciones realizadas son válidas para la búsqueda de vecinos sobre todo tipo de nubes LiDAR, y encontrar en qué condiciones aportan una mayor aceleración.

4.1. Entorno de pruebas y paralelización

En primer lugar, denotamos las posibles configuraciones de reordenación SFC [1] en las que se encuentra la nube y tipo de *Octree* que se construye sobre ella según el cuadro 4.1. Para comparar algoritmos de búsqueda, utilizaremos la notación del cuadro 4.2, teniendo en cuenta que *neighborsPtr* y *neighbors* no son más que las versiones para cada *Octree* del algoritmo básico de búsqueda visto en la sección 3.3.1.

Para la paralelización de las búsquedas, empleamos la librería OpenMP [28]. Como la búsqueda de vecinos, fijados un *kernel* y un radio, es independiente del centro elegido, realizamos una paralelización sencilla a nivel de bucle sobre todos los centros especificados en una lista v_c, véase algoritmo 3. En las pruebas de búsqueda de vecinos, distinguiremos dos casos:

[1] No construimos el *Octree* lineal sin haber reordenado previamente la nube, ya que la construcción de esa estructura depende intrínsicamente de las codificaciones SFC.

Cuadro 4.1: Combinaciones de *Octree* y SFC utilizadas en las pruebas.

Octree	**SFC**	**Código**
Punteros	Ninguno	`poct_unsorted`
	Morton	`poct_mort`
	Hilbert	`poct_hilb`
Lineal	Morton	`loct_mort`
	Hilbert	`loct_hilb`

Cuadro 4.2: Algoritmos de búsqueda utilizados en las pruebas.

Octree	**Algoritmo**
Punteros	*neighborsPtr*
Lineal	*neighbors*
	neighborsPrune
	neighborsStruct

Algoritmo 3 Búsqueda de vecindades paralela

Entrada: $\mathcal{P}$: nube de puntos
k: tipo de *kernel* (esfera, cubo, ...)
r: radio del *kernel*
v_c: array de centros de búsqueda
Salida: v_N: array de vecindades
μ: tamaño promedio

1: **procedure** PARALLELNEIGHSEARCH($\mathcal{P}, k, r, v_c$)
2: $\mu \leftarrow 0$
3: $num_searches \leftarrow size(v_c)$
4: `#pragma omp parallel for schedule(dynamic) reduction(+:`μ`)`
5: **for** $i \leftarrow 0 \ldots num_searches$ **do**
6: ▷ *neighSearch* es alguno de los algoritmos de búsqueda vistos.
7: $v_N[i] \leftarrow$ NEIGHSEARCH($P, k, r, v_c[i], root$)
8: $\mu \leftarrow \mu + size(v_N[i])$
9: **end for**
10: $\mu \leftarrow \mu / num_searches$
11: **return** (v_N, μ)
12: **end procedure**

1. Búsquedas aleatorias: v_c es un subconjunto de 10 000 puntos de $\mathcal{P}$, escogidos de forma aleatoria. Realizamos 5 repeticiones para cada medición con el mismo subconjunto, con el objetivo de minimizar la aleatoridad.

2. Búsqueda completa: v_c contiene a todos los puntos de $\mathcal{P}$, en el orden secuencial (tras el reordenamiento, si se ha realizado) de la nube.

El primer caso mide la eficiencia en el acceso aleatorio a los vecinos de un punto de la nube, útil para medir el rendimiento en aplicaciones donde se requiere calcular el vecindario de tan solo algunos puntos. El segundo caso aparece en muchos algoritmos prácticos donde es necesario calcular todos los vecinos de

todos los puntos. Las búsquedas completas se benefician aún más de la localidad de los puntos, ya que, gracias a la SFC, dos puntos próximos en el espacio estarán próximos en memoria, y por ende, también lo estarán sus vecindarios. De esta forma, al calcular el vecindario de un centro de v_c, estamos precalentando la caché con muchos de los nodos del *Octree* y los puntos a insertar que se utilizarán en los siguientes cálculos de vecindad. Para ambos casos, mostraremos los tiempos acumulados de ejecución de las búsquedas sobre el total de puntos de v_c, utilizando 40 hilos para la paralelización (excepto en la sección 4.5).

Almacenamos también el tamaño promedio de los vecindarios, μ, que será útil para comparar la escalabilidad de los métodos de búsqueda con respecto al radio y kernel elegidos. Para la distribución de las iteraciones, optamos por *dynamic scheduling* con tamaño de bloque automático, ya que, según nuestras pruebas, ofrece un mejor rendimiento en comparación con las demás estrategias de planificación (*static*, *guided*) disponibles en OpenMP. Todas nuestras pruebas fueron realizadas en la siguiente máquina de arquitectura NUMA, cuya arquitectura se puede consultar en más detalle en la figura 4.1:

Procesador: 4x Intel(R) Xeon(R) CPU E5-4620 v4 @ 2.10 GHz
Núcleos: 10 núcleos físicos por procesador (40 en total)
Arquitectura: Broadwell
Memoria RAM: 251 GiB
Sistema Operativo: Alma Linux 8.6 (Sky Tiger) Kernel 4.18.0
Compilador: GCC 12.1.1
Opciones de compilación: -O2

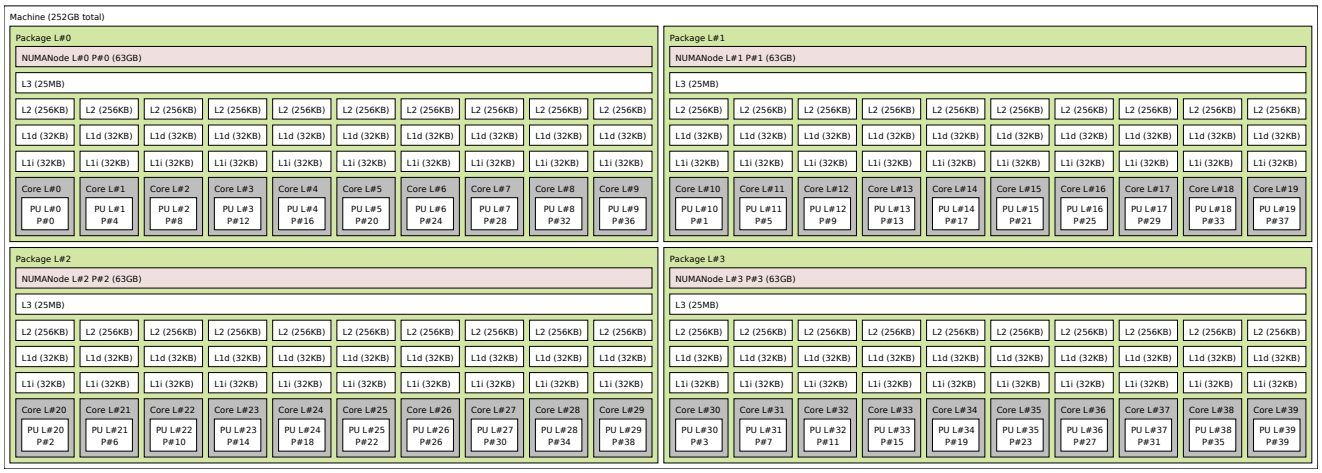


Figura 4.1: Diagrama de la arquitectura de la máquina NUMA.

4.2. Métodos de búsqueda exactos

Comenzamos analizando los algoritmos de búsqueda exacta, observando como afecta cada combinación de SFC, *Octree* y algoritmo de búsqueda a los tiempos de ejecución.

En primer lugar, analizaremos el rendimiento del *Octree* basado en punteros frente al del *Octree* lineal, equipado con el algoritmo de búsqueda *neighborsPrune*. Ambos métodos son directamente comparables al devolver el resultado en el mismo formato de array de punteros a cada punto del vecindario. También se ha comprobado que ambos métodos devuelvan el mismo resultado en cada prueba. En la figura 4.2 se resumen los resultados para la nube *Paris_Luxembourg_6* con búsquedas aleatorias, ejecutando sobre varios *kernels* y radios de búsqueda. Junto a los tiempos de ejecución, anotamos el tamaño promedio μ de las vecindades encontradas. Como ya hemos comentado, existe una relación directa entre este promedio y el tiempo de ejecución total de las búsquedas. En la figura 4.3, se muestran los tiempos de ejecución frente a μ agrupando en un solo gráfico todas las ejecuciones realizadas en varias nubes, radios y *kernels*.

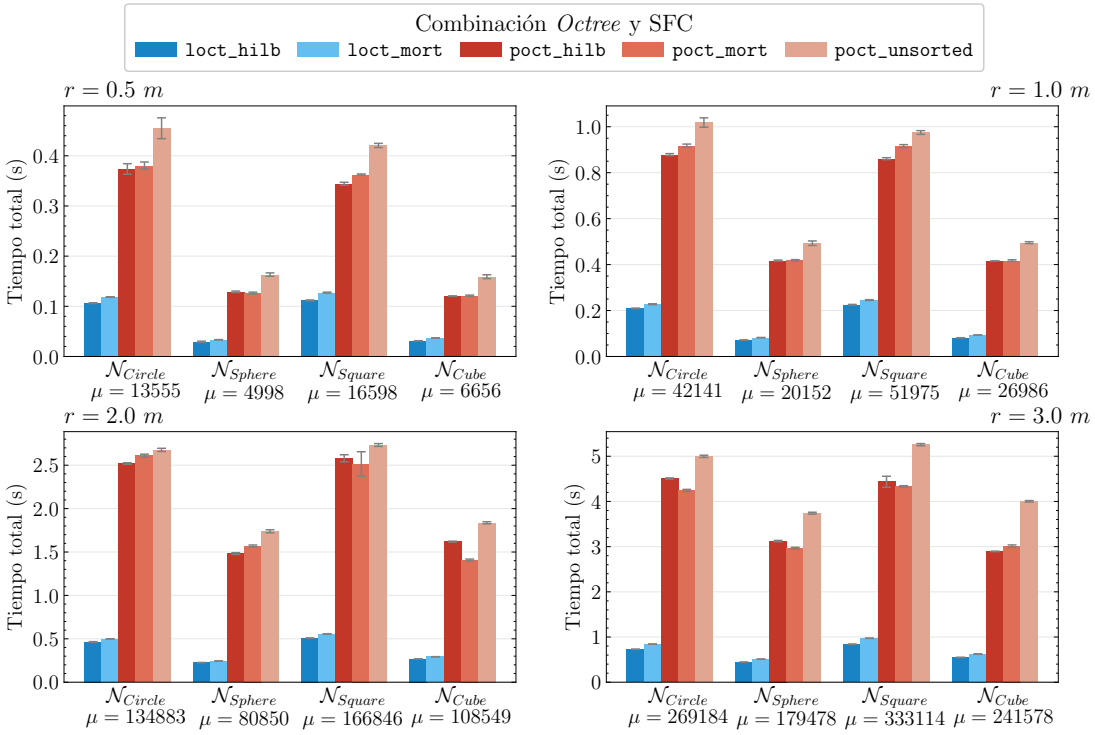


Figura 4.2: Tiempo de ejecución para 5000 búsquedas de vecinos con centros aleatorios en la nube *Paris_Luxembourg_6*. Algoritmo *neighborsPrune* usado en el *Octree* lineal.

Se aprecia que la estructura lineal supera con creces a la basada en punteros, especialmente cuando el radio de búsqueda es grande. Entre otros motivos, esto se debe a la eliminación de los pasos de dereferencia de punteros, a la inserción en bloque de puntos secuenciales en la vecindad gracias al método optimizado de búsqueda *neighborsPrune* y la reordenación por SFC. La diferencia entre los códigos de Morton y los de Hilbert es pequeña, y a lo largo de nuestras ejecuciones no encontramos un claro favorito, al variar la superioridad de uno frente al otro con cada nube, kernel, radio y número de hilos o estructura empleada.

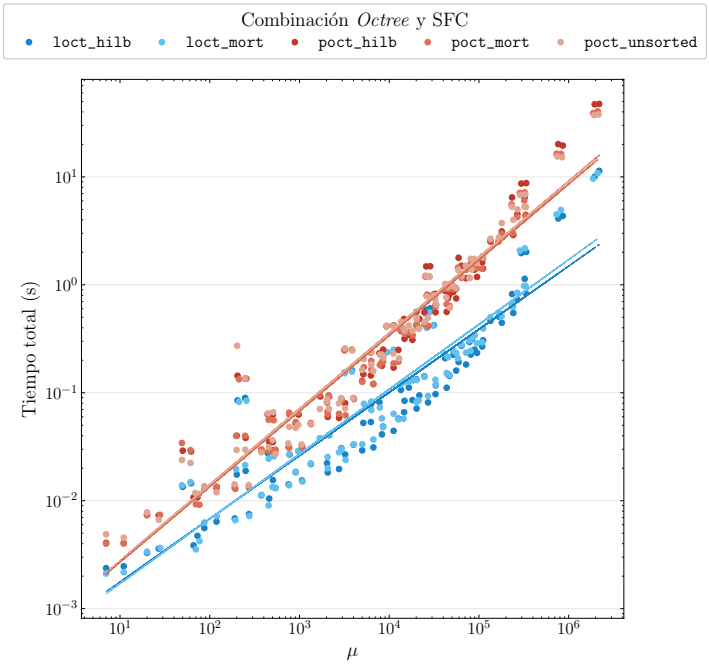


Figura 4.3: Nube de puntos en escala *log-log* mostrando la relación entre el tiempo de 5000 búsquedas aleatorias y μ en varias nubes y configuraciones. Algoritmo *neighborsPrune* utilizado en el *Octree* lineal.

El uso del método optimizado *neighborsStruct* del *Octree* lineal otorgará una mejora adicional. Como hemos discutido anteriormente, el tamaño promedio de las búsquedas μ puede escalar cúbicamente con el radio de búsqueda, pero este efecto se puede contrarrestar, en cierta medida, mediante el uso de la estructura basada en rangos. En la figura 4.4 comparamos los 4 algoritmos disponibles sobre búsquedas aleatorias en la nube *Paris_Luxembourg_6*, destacando la dominancia de *neighborsStruct* en las búsquedas con radio grande. Al igual que con la comparación de las combinaciones de *Octree* y SFC, podemos resumir nuestros resultados en una nube de puntos en escala logarítmica, véase figura 4.5.

En cuanto a las búsquedas completas, resumimos los resultados en el cuadro 4.3. Partiendo del algoritmo básico de búsqueda en un *Octree* basado en punteros,

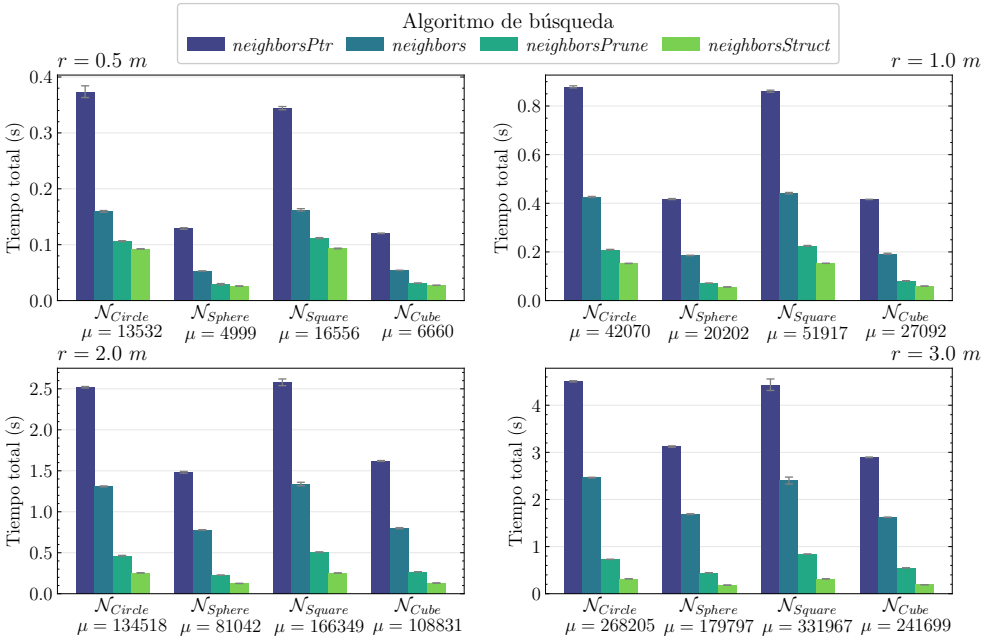


Figura 4.4: Tiempos acumulados para 5000 búsquedas de vecinos con centros aleatorios en la nube *Paris_Luxembourg_6* para varios algoritmos de búsqueda. La nube fue reordenada mediante la SFC de Hilbert.

sobre una nube sin reordenar, hemos llegado al algoritmo *neighborsStruct* que, junto con el reordenamiento mediante SFC de Morton o de Hilbert, acelera las búsquedas de manera muy significativa. Esta mejora en el rendimiento llega a ser de más de un orden de magnitud en las ejecuciones cuando μ es grande y, según nuestras pruebas, el tiempo de acceso a los vecinos dentro de la estructura basada en rangos no es mucho mayor que el tiempo de acceso a través de un vector de punteros a cada punto.

4.3. Comparación con otras estructuras

Una vez establecida la superioridad del método *neighborsStruct* del *Octree* lineal sobre los otros desarrollados, comparamos los resultados obtenidos con otras estructuras *Octree*. En particular, estudiaremos la estructura desarrollada por Behley et al. [4], que llamaremos *unibnOctree*, y el *Octree* de la popular librería de procesamiento de nubes de puntos *PCL* [32]. Este último *Octree* presenta algunos problemas de comparación debido a que su construcción acepta un parámetro de tamaño de octante mínimo (resolución), frente al número de puntos máximo N_{max} que usamos en nuestro desarrollo. Sin embargo, la estructura *unibnOctree*, que es un *Octree* basado en punteros con algunas optimizaciones, presenta las siguientes

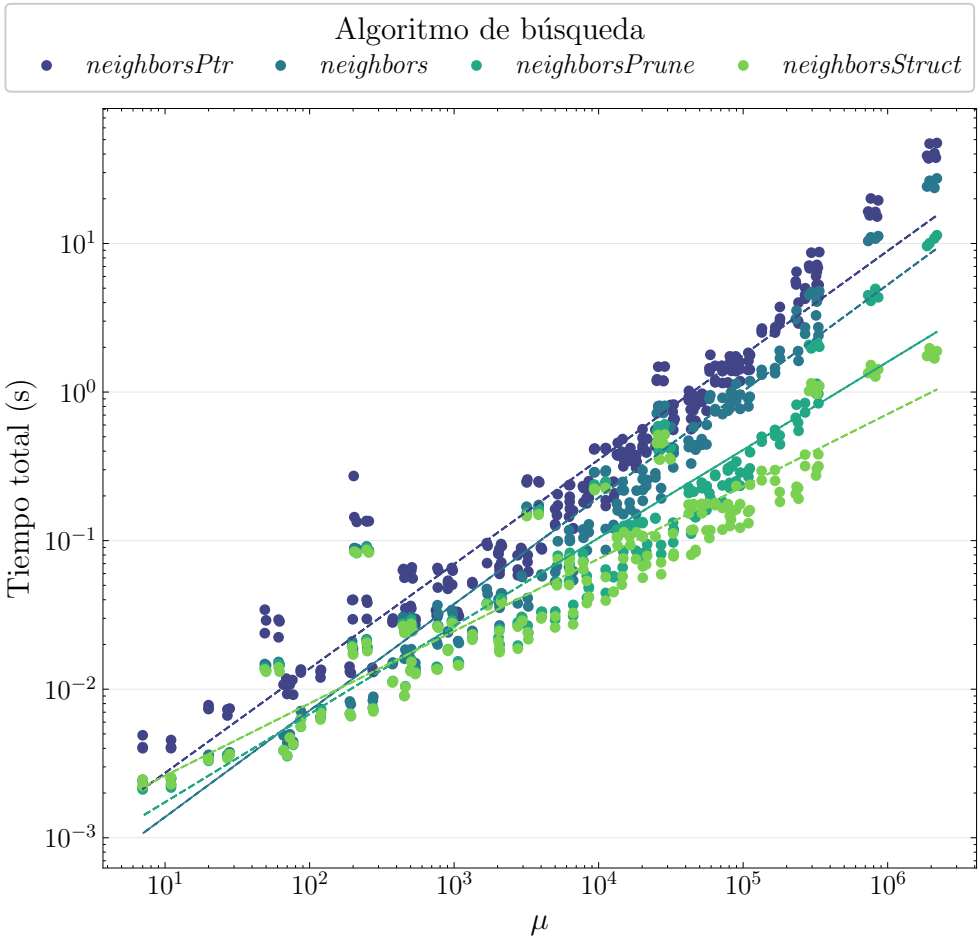


Figura 4.5: Nube de puntos en escala *log-log* mostrando la relación entre el tiempo de 5000 búsquedas con centros aleatorios y los distintos algoritmos de búsqueda utilizados. La nube fue reordenada mediante la SFC de Morton.

similitudes con nuestro *Octree* lineal:

- Ambas estructuras se centran en búsquedas de radio fijo y tienen interfaces similares para los kernels $\mathcal{N}_{Sphere}$ y $\mathcal{N}_{Cube}$.
- Ambas estructuras aprovechan las SFCs para la poda e inserción en bruto de puntos dentro de un octante. Sin embargo, *unibnOctree* no utiliza una estructura basada en rangos para el resultado.
- Nuestras comprobaciones geométricas para el kernel $\mathcal{N}_{Sphere}$ en el *Octree* lineal fueron directamente adaptadas del *unibnOctree*.

Por tanto, debe tomarse como la comparación más directa de esta sección. También contrastaremos el rendimiento con la estructura *kd*-tree de la librería

Cuadro 4.3: Tiempos de ejecución de búsquedas de vecinos sobre todos los puntos de la nube con *Sphere kernel* y con SFC de Morton.

Nube	r (m)	μ	Algoritmo	Tiempo (s)
Lille_0	3.0	71483	• *neighborsPtr*	273.66
			• *neighbors*	159.97
			• *neighborsPrune*	101.58
			• *neighborsStruct*	**35.46**
Paris_Luxembourg_6	3.0	179181	• *neighborsPtr*	1037.23
			• *neighbors*	691.14
			• *neighborsPrune*	354.35
			• *neighborsStruct*	**56.37**
5080_54400	10.0	12587	• *neighborsPtr*	49.20
			• *neighbors*	43.78
			• *neighborsPrune*	37.73
			• *neighborsStruct*	**27.37**
sg27_station8	0.1	14526	• *neighborsPtr*	1663.72
			• *neighbors*	1303.94
			• *neighborsPrune*	965.05
			• *neighborsStruct*	**403.00**
Speulderbos	0.25	4658	• *neighborsPtr*	1126.93
			• *neighbors*	936.02
			• *neighborsPrune*	805.68
			• *neighborsStruct*	**476.36**

PCL (originalmente del proyecto *FLANN* [27]), con el objetivo de analizar si en algún caso esta es más adecuada que los *Octrees*. Las pruebas siguen el mismo formato de antes: presentamos en la figura 4.6 la nube de tiempos en escala logarítmica comparando los tiempos de ejecución de *unibnOctree* y *neighborsStruct* para búsquedas aleatorias, aunque esta vez tan solo mostrando el kernel $\mathcal{N}_{Sphere}$, que es el único compatible con todas las estructuras (*unibnOctree* también soporta $\mathcal{N}_{Cube}$ y distancias L^1, en cambio PCL solo kernels esféricos).

Observamos que, para radios suficientemente grandes, el *Octree* lineal obtiene un tiempo de ejecución alrededor de 2-3 veces más rápido que *unibnOctree*. Además, es el método que mejor escala respecto al tamaño del kernel, lo cual se refleja en una menor pendiente en la recta de regresión. En el cuadro 4.4a resumimos estas mediciones, anotando el *speedup* obtenido frente al *Octree* de PCL. En el cuadro 4.4b hacemos lo mismo para los datos obtenidos en búsquedas completas. El *kd*-tree no tiene un buen rendimiento para búsquedas en radio fijo, especialmente sobre centros aleatorios, siendo esta estructura más comúnmente utilizada en búsquedas k-NN exactas o aproximadas [27].

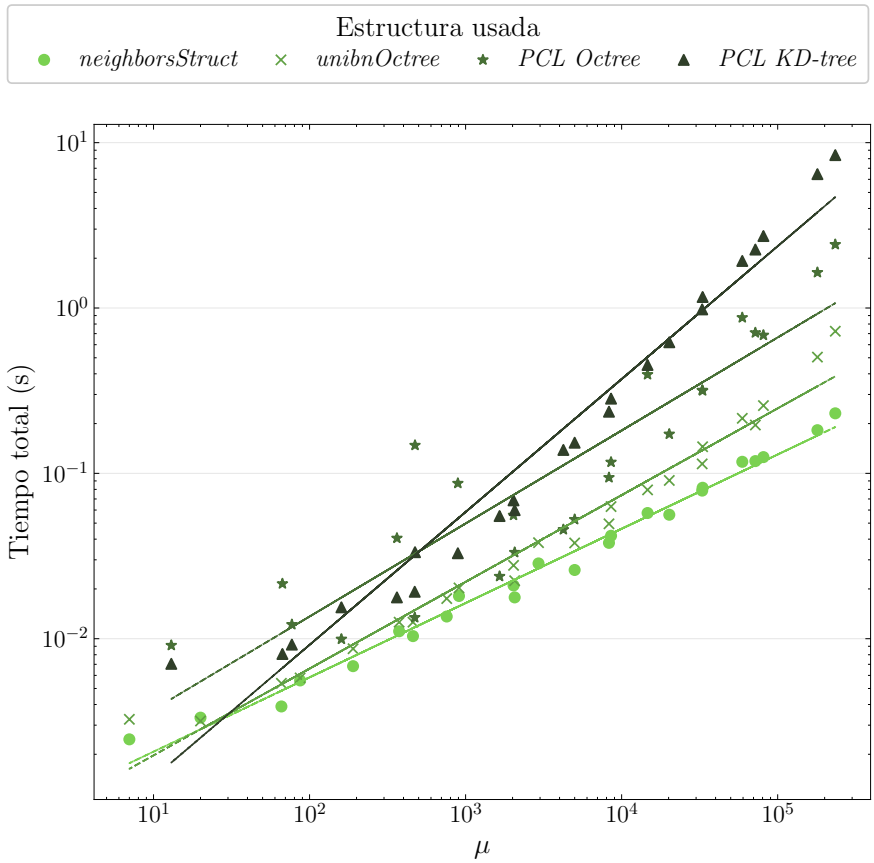


Figura 4.6: Comparación del rendimiento de distintas estructuras en búsquedas aleatorias en $\mathcal{N}_{Sphere}$. La nube fue reordenada mediante la SFC de Hilbert.

Cuadro 4.4: Comparación de tiempos de ejecución promedio (en segundos) sobre varios datasets por rangos de μ. Kernel esférico y reordenación de Hilbert usados.

(a) Búsquedas aleatorias

Rango μ	*PCL Octree*	*neighborsStruct*	*unibnOctree*	*PCL-KDtree*
$[10^0, 10^2)$	0.01	0.00 (**3.74x**)	0.00 (3.24x)	0.01 (1.76x)
$[10^2, 10^4)$	0.06	0.02 (**2.83x**)	0.03 (2.13x)	0.09 (0.65x)
$[10^4, 10^6]$	0.84	0.12 (**7.18x**)	0.26 (3.24x)	2.78 (0.30x)

(b) Búsquedas completas

Rango μ	*PCL Octree*	*neighborsStruct*	*unibnOctree*	*PCL-KDtree*
$[10^0, 10^2)$	17.03	6.18 (2.76x)	5.88 (**2.90x**)	12.75 (1.34x)
$[10^2, 10^4)$	1046.52	101.06 (**10.36x**)	197.62 (5.30x)	463.51 (2.26x)
$[10^4, 10^6)$	1336.45	131.61 (**13.75x**)	231.41 (7.82x)	1808.98 (1.35x)

4.4. Búsquedas aproximadas

En esta sección compararemos la mejora en tiempos de búsqueda, junto a la pérdida de precisión, al emplear búsquedas aproximadas (método *neighborsApprox*) en el *Octree* lineal. Para visualizar esto, empleamos un diagrama de dispersión entre el tiempo empleado y el porcentaje de puntos encontrados, siendo el 100 % el μ de las búsquedas exactas (realizadas con *neighborsStruct*). En la figura 4.7 se resumen los resultados para la nube *Lille_0*. El uso de búsquedas aproximadas acelera el proceso, pudiendo ser el rango de tolerancias $5\,\% - 25\,\%$ ideal para muchas aplicaciones de alto rendimiento donde el conjunto exacto de puntos no es necesario. Sin embargo, un problema persistente y no despreciable en las búsquedas aproximadas es el sesgo geométrico causado por incluir todos los puntos dentro de un octante, cuyos bordes son planos y no respetan la geometría real de la superficie capturada.

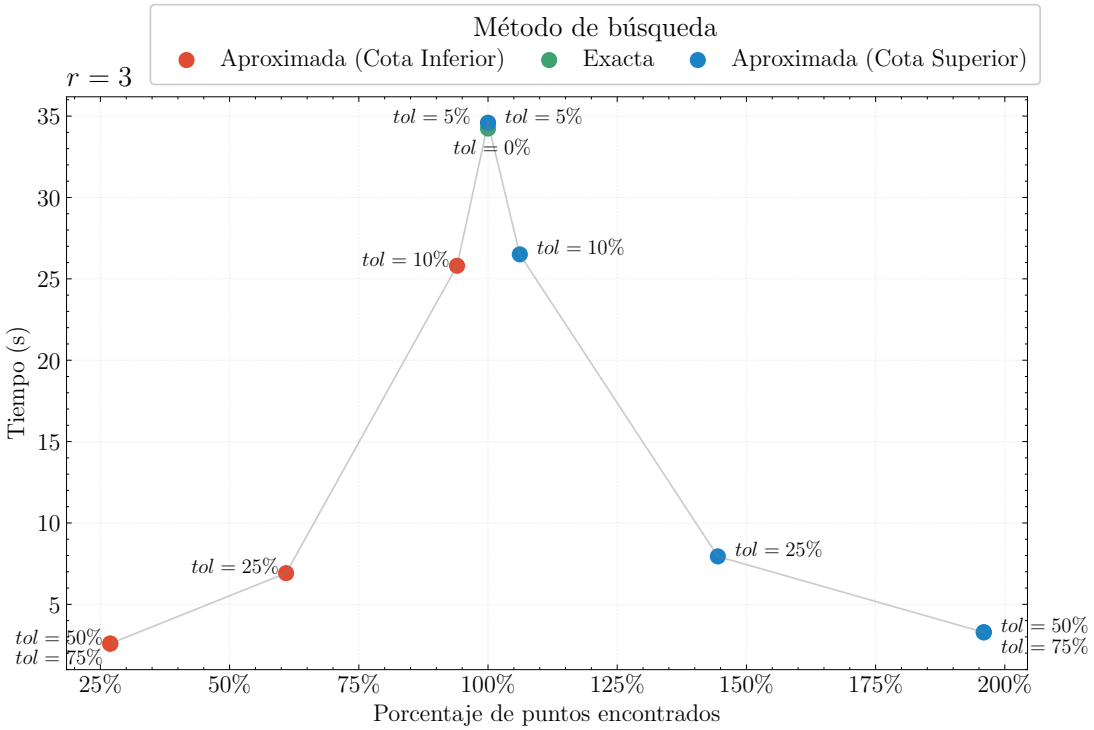
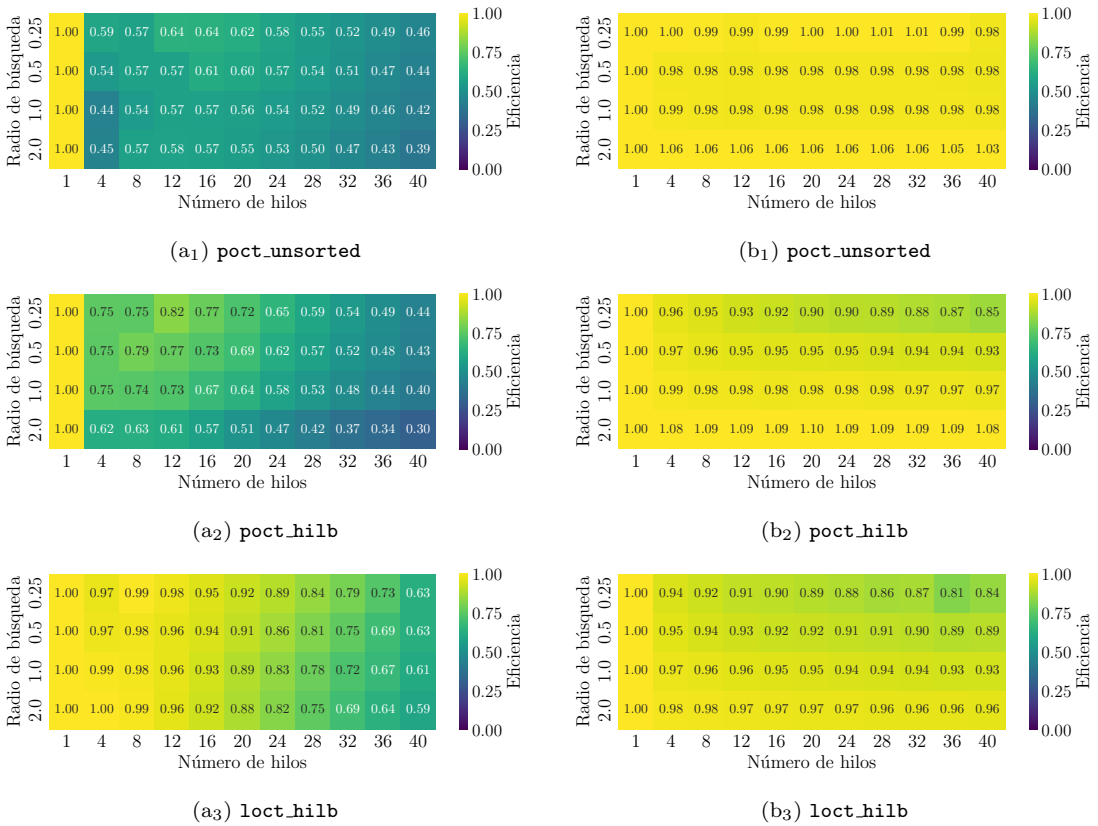


Figura 4.7: Comparación en la aproximación de búsquedas completas en la nube *Lille_0*. La nube fue ordenada mediante la SFC de Hilbert.

4.5. Eficiencia en el paralelismo

Se ha realizado un análisis de escalabilidad computacional con el objetivo de comprobar si el *Octree* elegido (basado en punteros o lineal) o la SFC afectan a la eficiencia paralela. En los mapas de calor de la figura 4.8a se muestra la eficiencia en la paralelización de las búsquedas para el dataset *Lille_0*, para diferentes combinaciones de *Octree* y reordenamiento usando un subconjunto de centros aleatorio. Al igual que en los tiempos de búsqueda, debemos tratar el cálculo de vecindades sobre toda la nube por separado. En la figura 4.8b mostramos este caso. Como vemos, la paralelización es casi ideal en los casos de búsqueda completa, mientras que en las búsquedas aleatorias vemos una mejora considerable al usar el *Octree* lineal.



(a$_1$) `poct_unsorted` (b$_1$) `poct_unsorted`

(a$_2$) `poct_hilb` (b$_2$) `poct_hilb`

(a$_3$) `loct_hilb` (b$_3$) `loct_hilb`

(a) Búsquedas aleatorias (b) Búsquedas completas

Figura 4.8: Matrices de eficiencia para la paralelización de búsquedas en el dataset *Lille_0*. Algoritmo *neighborsPrune* usado en el *Octree* lineal.

4.6. Reordenamiento y construcción de *Octrees*

En esta sección analizaremos los tiempos de preprocesamiento necesarios antes de ejecutar cada una de las configuraciones de *Octree* y SFC descritas. Comenzaremos con el cálculo de la bounding box $\mathcal{B}$, la codificación de puntos según la SFC de Morton o de Hilbert y el reordenamiento de la nube. Para la paralelización de cada uno de estos pasos, utilizamos las siguientes técnicas:

1. Cálculo de $\mathcal{B}$: asignación de variables locales a cada hilo con los mínimos y máximos encontrados para cada coordenada, más un paso de reducción secuencial.

2. Codificación de puntos de la nube: paralelización simple a nivel de bucle.

3. Reordenamiento de la nube: algoritmo *radix sort* paralelo, utilizando 4 pases de 16 bits cada uno sobre los códigos de 64 bits.

Además, también hemos paralelizado la lectura de los archivos *.las* en los que se encuentran nuestras nubes, que alcanzan tamaños de decenas de GB. En el cuadro 4.5 resumimos los resultados obtenidos. Nótese que los tiempos de reordenación también se dan por separado para cada SFC, ya que se recomputan las codificaciones en cada paso de *radix sort* para no tener que guardarlas en un array adicional, lo cuál es más lento para ambas curvas en nuestra implementación. Observamos una diferencia considerable en ambos pasos al usar curvas de Hilbert. Esto sugiere que en casos en los que se necesite una construcción rápida sobre nubes grandes podría ser más adecuado usar curvas de Morton.

Cuadro 4.5: Tiempos de cálculo de la codificación y reordenamiento sobre varias nubes y SFCs.

Nube (Tamaño)	SFC	Codificación (s)	Reordenamiento(s)
Lille_0 (10M)	Morton	0.03	0.82
	Hilbert	0.08	1.01
station1 (50M)	Morton	0.25	3.36
	Hilbert	0.45	4.49
Speulderbos (721M)	Morton	3.57	50.26
	Hilbert	6.75	68.65

Por otro lado, recopilamos varias mediciones de los tiempos de construcción de cada *Octree* en el cuadro 4.6. El *Octree* lineal es un claro ganador, al beneficiarse de una construcción paralelizable y que no necesita efectuar tantas reservas y

movimientos en memoria como las versiones basadas en punteros. Además del tipo de árbol elegido, el parámetro que más afecta al tiempo de construcción es el umbral de puntos en las hojas N_{max}, ya que cuanto más bajo sea este, más nodos tendrá el árbol.

También es interesante que el reordenamiento por SFCs aumente la velocidad de construcción del árbol en el *Octree* basado en punteros (así como el *unibnOctree*, que omitimos en la tabla), que no usa los códigos para su construcción. Al igual que en las búsquedas de vecinos, los efectos derivados de la mejora en localidad bastan para acelerar el proceso, sin necesidad de modificar los algoritmos ni la estructura construida sobre la nube.

Cuadro 4.6: Tiempos de construcción de los *Octrees* sobre varias nubes en segundos. Notación: t_h tiempo de construcción de las hojas, t_i de la parte interna y t_t tiempo total, en segundos.

		$N_{max} = 64$			$N_{max} = 128$		
Nube (Tamaño)	*Octree*	t_h	t_i	t_t	t_h	t_i	t_t
Lille_0 (10M)	`poct_unsorted`	—	—	2.34	—	—	1.69
	`poct_hilb`	—	—	2.43	—	—	1.05
	`unibn_hilb`	—	—	0.91	—	—	0.70
	`loct_hilb`	0.14	0.09	0.24	0.07	0.04	0.11
station1 (50M)	`poct_unsorted`	—	—	12.2	—	—	8.75
	`poct_hilb`	—	—	9.36	—	—	7.47
	`unibn_hilb`	—	—	6.38	—	—	4.88
	`loct_hilb`	0.65	0.68	1.34	0.38	0.30	0.68
Speulderbos (721M)	`poct_unsorted`	—	—	330	—	—	196
	`poct_hilb`	—	—	118	—	—	95.5
	`unibn_hilb`	—	—	88.2	—	—	84.6
	`loct_hilb`	9.0	10.2	19.2	7.3	4.8	12.2

Por último, realizamos un análisis del consumo de memoria adicional necesario para los *Octrees*. Definimos T_o como el tamaño de cada nodo de los *Octrees* y T_p como el tamaño de memoria adicional que necesitan por cada punto insertado en ellos. Tenemos los siguientes valores:

- *Octree* basado en punteros: $T_o = 120$ bytes y $T_p = 8$ bytes (se almacenan las direcciones de cada punto en el árbol).
- *Octree* lineal: $T_o = 52$ bytes (se puede reducir a 28 si no se precalculan los centros y se mantiene el array *prefix*) y $T_p = 0$.

- *unibnOctree*: $T_o = 128$ bytes y $T_p = 4$ bytes (se almacena un índice de 4 bytes por punto).

Para nuestros datasets, con $N_{max} = 128$, hemos medido un porcentaje promedio de nodos respecto a puntos $\rho = 0.045$ para el *Octree* basado en punteros y el lineal (completos, todos los nodos internos tienen 8 hijos) y $\rho = 0.029$ para el *unibnOctree* (no completo) . Teniendo en cuenta que nuestros puntos tienen 64 bytes (32 bytes en `Point` y 32 bytes en `PointMetadata`), el uso total de memoria aproximado sería $N(64 + T_p + T_o\rho)$. Podemos calcular el gasto extra de memoria para cada *Octree* como

$$\omega = \frac{T_p + T_o\rho}{64}. \tag{4.1}$$

De esta forma, para el *Octree* basado en punteros $\omega = 20.93\,\%$, para el *Octree* lineal $\omega = 3.6\,\%$ y para el *unibnOctree* $\omega = 12.05\,\%$. Estos porcentajes fueron medidos experimentalmente y se apreciaron variaciones muy pequeñas, de no más del $2\,\%$ en ninguna nube u *Octree*. Como vemos, el *Octree* lineal tiene el menor impacto en memoria, gracias a que no necesita guardar ningún tipo de información por punto debido a su construcción basada en rangos de SFC, representada de forma compacta en los arrays *offsets* y *layoutRanges*.

Capítulo 5

Discusión de los resultados

5.1. Conclusiones obtenidas

Como hemos ido apuntando a lo largo del trabajo, no es sencillo trabajar con nubes de puntos LiDAR debido a su alta irregularidad y sus diferentes modos de captura (por ejemplo, las nubes aéreas y terrestres presentan notables diferencias). En este trabajo hemos presentado un método lo más general posible para acelerar las búsquedas de vecinos y, para poder demostrar su eficiencia, ha sido necesario realizar un gran conjunto de pruebas, con miles de mediciones y millones de búsquedas individuales, cubriendo una gran parte de las configuraciones posibles. Sintetizando todo lo visto en el anterior capítulo, podemos extraer las siguientes conclusiones generales:

1. El reordenamiento mediante curvas de Morton y de Hilbert mejora la localidad de la nube en memoria, lo cual reduce el tiempo de ejecución de las búsquedas de vecinos, así como el proceso de construcción del propio *Octree*.

2. Las SFCs de Morton y de Hilbert aportan un rendimiento similar en la mayoría de las nubes, siendo en ciertas ocasiones una de ellas preferible sobre la otra. El aumento de tiempo de codificación de la SFC de Hilbert frente a la de Morton no suele ser demasiado relevante frente al tiempo de las búsquedas, excepto en nubes grandes y radios de búsqueda pequeños.

3. Las implementaciones de búsqueda de vecinos especializadas *neighborsPrune* y *neighborsStruct* aportan una mejora muy considerable frente al *Octree* basado en punteros. Especialmente, *neighborsStruct* escala mucho mejor que el algoritmo clásico al aumentar el radio de búsqueda.

4. Las búsquedas aproximadas, que aprovechan aún más la estructura basada en rangos para el resultado y la geometría del *Octree*, pueden obtener vecindarios en un tiempo mucho menor, a costa de una pérdida de precisión en el resultado.

5.2. Comparación con el estado del arte

Además de todo lo anterior, también podemos decir que hemos superado el rendimiento de las estructuras encontradas durante el estudio del estado del arte. En concreto hemos obtenido un tiempo de ejecución de 2 a 10 veces más rápido, en general, que las estructuras *Octree* implementadas por Behley et al. [4] y PCL [32]. Nuestra propuesta combina la estructura de *Octree* lineal, propuesta por Keller et al. [23], el uso de curvas SFC de Morton y Hilbert para el reordenamiento de la nube y las optimizaciones basadas en la poda del árbol durantes las búsquedas de vecinos propuestas por Behley et al. [4].

Todo esto, junto a la estructura basada en rangos de índices en la que expresamos los resultados, logra una mayor eficiencia para la búsqueda de vecindarios con radio fijo en todas las nubes de puntos LiDAR que probamos. Como hemos visto en la sección 4.3, el método *neighborsStruct* del *Octree* lineal escala mejor con el radio de búsqueda (o número promedio de puntos encontrados) y es más rápido que el *unibnOctree* incluso cuando este es bajo.

Finalmente anotamos que puso una gran atención al detalle en la construcción paralela y la optimización en memoria de la estructura lineal, obteniendo un *Octree* rápido y ligero, adecuado para todo tipo de entornos y tamaños de nube. Presenta también un buen coeficiente de paralelización, incluso en búsquedas sobre centros aleatorios con un número de hilos elevado, donde ocurren más fallos caché.

Capítulo 6

Conclusiones y posibles ampliaciones

A lo largo de este trabajo, hemos expuesto dos mejoras sobre la estructura clásica de *Octree* basado en punteros para la búsqueda de vecinos en nubes de puntos LiDAR: el proceso de reordenamiento mediante curvas de llenado del espacio tridimensionales y el uso de una estructura lineal junto a algoritmos de búsqueda optimizados. Ambos métodos han reducido los tiempos de ejecución en los datasets y *kernels* de búsqueda probados. Una parte de los métodos propuestos y los resultados obtenidos han sido presentados y publicados en las XXXV Jornadas de Paralelismo SARTECO, en el artículo titulado "Aplicación de curvas de llenado del espacio en nubes de puntos 3D almacenadas en Octrees".

No obstante, aún quedan muchas cuestiones abiertas. Por ejemplo, no hemos explorado en detalle por qué las SFCs son más beneficiosas en algunas nubes que en otras, y con qué métodos se puede cuantificar la mejora en la localidad espacial de la nube. Un estudio posterior podría medir el número de fallos caché durante las búsquedas, así como calcular una métrica que defina la localidad espacial de la nube. La optimización del proceso de reordenamiento también puede ser de interés. Por ejemplo, el algoritmo *radix sort* se puede ejecutar solo hasta cierto nivel $l < 21$. Si l continúa siendo mayor que el nivel máximo de subdivisión alcanzado por los *Octrees*, el rendimiento podría ser similar.

Por otro lado, algunas nubes, como las captadas por sensores LiDAR aéreos, son generalmente planas. En estas, podría ser adecuado el uso de SFCs bidimensionales en vez de tridimensionales. Estas SFCs son más sencillas de calcular y pueden representar más niveles en un entero de 64 bits, lo cuál podría ser de utilidad en nubes de gran tamaño y densidad. Esta idea se puede llevar más allá y utilizar estructuras que tengan en cuenta la estructura generalmente plana del

terreno, ignorando o tratando de una forma diferente a la dimensión vertical.

También es posible la construcción del *Octree* lineal, así como las búsquedas de vecinos, en otras arquitecturas paralelas. La publicación original de Keller at al. [23] utiliza una arquitectura dual GPU/CPU mediante el uso de CUDA y la librería Thrust [10]. Además de GPUs, también podría ser objeto de investigación el uso de librerías como MPI para la distribución de las búsquedas o la construcción de árboles masivos, analizando como fragmentar el árbol entre los nodos y cómo generalizar los algoritmos de búsqueda en ellos.

Apéndice A

Manuales técnicos

A.1. Archivos fuente y headers principales

La mayor parte del programa [13], debido a estar generalizado mediante templates (para el tipo de punto y de *Octree*), se construyó en archivos de header, dejando tan solo archivos `.cpp` para el programa principal, para el *Octree* basado en punteros (ya que la implementación original estaba en ese formato) y para algunos otras partes más secundarias. Las partes del programa de mayor interés son las siguientes:

1. `src/main.cpp` Programa principal.
2. `[inc/src]/main_options.[h/c]pp` Almacenamiento de argumentos y configuración del programa principal.
3. `inc/Geometry/...` Estas clases definen los tipos de punto LiDAR utilizados, así como las bounding boxes y métodos para obtenerlas.
4. `inc/NeighborKernels/...` Define los kernels de búsqueda y los métodos para determinar si un octante (o un punto) los interseca, está contenido o está fuera de ellos.
5. `inc/PointEncoding/...` Define las SFCs de Morton (mediante libmorton [3]) y de Hilbert (mediante la implementación de [25]). Incluye métodos para el reordenamiento de la nube, el cálculo paralelo de los códigos y otras muchas funciones útiles durante la construcción del *Octree* lineal.
6. `inc/readers/...` Define las clases para leer las nubes de puntos a partir de archivos *.las* y *.txt*.

7. `inc/linear_octree.hpp` Implementa el *Octree* lineal y sus algoritmos de búsqueda optimizados.

8. `inc/neighbor_set.hpp` Implementa la estructura que retiene los resultados de *neighborStruct*.

9. `inc/octree_benchmark.hpp` Sirve de punto de entrada para las pruebas sobre toda clase de *Octrees* (y el *kd*-tree de PCL).

10. `[inc/src]/octree.[h/c]pp` Implementa el *Octree* basado en punteros.

11. `inc/unibnOctree.hpp` Implementa el *unibnOctree*, código de Behley et al. [4].

12. `inc/search_set.hpp` Construye los conjuntos de búsqueda sobre los que se ejecutan las pruebas.

A.2. Instrucciones de compilación

La compilación del programa se hace mediante CMake que, además de las librerías esenciales para el desarrollo en C++, tiene las siguientes dependencias:

- *OpenMP* [28], para la paralelización de las búsquedas, la lectura de puntos, la construcción del árbol, las codificaciones y el reordenamiento de la nube.
- *LASTools* [20, 21], para la lectura de archivos *.las*.
- *libmorton* [3], ya incluido en *inc/PointEncoding/libmorton*, para la codificación rápida de Morton.
- (OPCIONAL) *PCL* [32] versión 1.15. Debe estar instalada en *~/local/pcl* , aunque se puede modificar la versión y el path en *CMakeLibraries.cmake*. Si no se encuentra PCL, se puede compilar el programa de igual forma (la opción de compilador `HAVE_PCL` será falsa).

En el archivo *README.md* se pueden encontrar instrucciones detalladas de como instalar cada una de estas dependencias. Una vez instaladas, se puede compilar el programa mediante A.1.

```
cmake -B build -DCMAKE_BUILD_TYPE=Release .
cmake --build build
```

Listing A.1: Compilación del programa principal *octree-benchmarks*.

A.3. Script utilizado en las pruebas

A continuación, incluimos el script de Bash A.2, utilizado para obtener todos los resultados expuestos en el capítulo 4 (también disponible en el código, archivo *bench_executions_tfg.bash*). Cabe destacar que algunas de estas ejecuciones dependen de si la librería PCL fue encontrada durante la compilación. En el caso de que no fuera encontrada pero se quisieran repetir los experimentos que no comparan con el *Octree* y el *kd*-tree de PCL, se deben eliminar las menciones a *neighborsPCLKD* y *neighborsPCLOct*. Ver Apéndice B para más información acerca de cada una de las opciones.

```
# Datasets utilizados
datasets_low_density=(
    "data/paris_lille/Lille_0.las"
    "data/dales_las/test/5080_54400.las"
    "data/paris_lille/Paris_Luxembourg_6.las"
)
datasets_high_density=(
    "data/semantic3d/bildstein_station1_xyz_intensity_rgb.las"
    "data/semantic3d/sg27_station8_intensity_rgb.las"
    "data/speulderbos/Speulderbos_2017_TLS.las"
)
datasets_build=(
    "data/paris_lille/Lille_0.las"
    "data/semantic3d/station1_xyz_intensity_rgb.las"
    "data/speulderbos/Speulderbos_2017_TLS.las"
)

# Variables comunes
FOLDER="out_tfg_v3"
THREADS="1,4,8,12,16,20,24,28,32,36,40"
KERNELS_3D="cube,sphere"
N_SEARCHES="10000"
TOLERANCES="5.0,10.0,25.0,50.0,75.0"
NUM_THREADS="1,4,8,12,16,20,24,28,32,36,40"
FULL_NO_PCLKD="neighbors,neighborsPrune,neighborsPtr,neighborsStruct"
FULL_ALGOS="neighbors,neighborsPrune,neighborsPtr,neighborsStruct,neighborsPCLKD
    ,neighborsUnibn,neighborsPCLOct"
set -e
mkdir -p "$FOLDER"

# Busquedas aleatorias
for data in "${datasets_low_density[@]}"; do
  ./build/octrees-benchmark --kernels "all" -i "$data" -o "$FOLDER/subset" -r "
      0.5,1.0,2.0,3.0" -s "$N_SEARCHES" --repeats 5 -a "neighborsPCLOct"
done
for data in "${datasets_high_density[@]}"; do
  ./build/octrees-benchmark --kernels "all" -i "$data" -o "$FOLDER/subset" -r "
      0.01,0.05,0.1,0.2" -s "$N_SEARCHES" --repeats 5 -a "neighborsPCLOct"
done

# Busquedas completas
./build/octrees-benchmark --kernels "sphere" -i "data/semantic3d/
    bildstein_station1_xyz_intensity_rgb.las" -o "$FOLDER/full" -r "
    0.01,0.05,0.1" -s "all" --sequential --repeats 1 --no-warmup -a "
    neighborsPCLOct"
./build/octrees-benchmark --kernels "sphere" -i "data/dales_las/test/5080_54400.
    las" -o "$FOLDER/full" -r "15.0" -s "all" --sequential --repeats 1 --no-
    warmup -a "neighborsPCLOct"
```

```
./build/octrees-benchmark --kernels "sphere" -i "data/paris_lille/Lille_0.las" -
    o "$FOLDER/full" -r "3.0" -s "all" --sequential --repeats 1 --no-warmup -a "
    neighborsPCLOct"
./build/octrees-benchmark --kernels "sphere" -i "data/paris_lille/
    Paris_Luxembourg_6.las" -o "$FOLDER/full" -r "3.0" -s "all" --sequential --
    repeats 1 --no-warmup -a "neighborsPCLOct"
./build/octrees-benchmark --kernels "sphere" -i "data/semantic3d/
    sg27_station8_intensity_rgb.las" -o "$FOLDER/full" -r "0.01,0.025,0.05" -s "
    all" --sequential --repeats 1 --no-warmup -a "neighborsPCLOct"
./build/octrees-benchmark --kernels "sphere" -i "data/speulderbos/
    Speulderbos_2017_TLS.las" -o "$FOLDER/full" -r "0.25" -s "all" --sequential
    --repeats 1 --no-warmup -a "neighborsPCLOct"

# Busquedas aproximadas
./build/octrees-benchmark --kernels "sphere" -i "data/paris_lille/Lille_0.las" -
    o "$FOLDER/approx_full" -r "3.0" -s "all" --sequential --repeats 1 --no-
    warmup -a "neighborsStruct,neighborsApprox" --approx-tol "$TOLERANCES"

# Paralelismo
./build/octrees-benchmark --kernels "sphere" -i "data/paris_lille/Lille_0.las" -
    o "$FOLDER/parallel_subset" -r "0.25,0.5,1.0,2.0" -s "$N_SEARCHES" --repeats
     5 -a "neighborsPtr,neighborsStruct" --num-threads "$NUM_THREADS"
./build/octrees-benchmark --kernels "sphere" -i "data/paris_lille/Lille_0.las" -
    o "$FOLDER/parallel_full" -r "0.25,0.5,1.0,2.0" -s "all" --sequential --
    repeats 1 --no-warmup -a "neighborsPtr,neighborsStruct" --num-threads "
    $NUM_THREADS"

# Tiempo de codificacion, reordenamiento y construccion de Octrees
for data in "${datasets_build[@]}"; do
    ./build/octrees-benchmark -i "$data" -o "$FOLDER/build_128" --debug --max-
        leaf 128 --repeats 3
    ./build/octrees-benchmark -i "$data" -o "$FOLDER/build_64" --debug --max-
        leaf 64 --repeats 3
done
```

Listing A.2: Script de pruebas usado para la evaluación experimental.

Apéndice B

Manuales de usuario

En el cuadro B.1 describimos las opciones principales del programa, tales como los archivos de entrada/salida, cómo configurar la ejecución de los algoritmos y los radios y kernels de búsqueda seleccionados. En el cuadro B.2, definimos algunas opciones más secundarias o específicas de ciertas pruebas.

Cuadro B.1: Opciones principales del programa

Opción	Descripción
`-h`, `--help`	Muestra el mensaje de ayuda.
`-i`, `--input`	Ruta al archivo de entrada.
`-o`, `--output`	Ruta al archivo de salida.
`-r`, `--radii`	Radios para los benchmarks (separados por comas, por ejemplo $2.5, 5.0, 7.5, 10.0$).
`-s`, `--searches`	Número de búsquedas a realizar (centros aleatorios, a menos que se use `--sequential`). Usa `all` para buscar sobre toda la nube.
`-t`, `--repeats`	Número de repeticiones por benchmark.
`-k`, `--kernels`	Núcleos espaciales a usar (separados por comas o `all`). Valores posibles: `sphere`, `cube`, `square`, `circle`.
`-a`, `--search-algo`	Algoritmos de búsqueda (separados por comas o `all`). Valores posibles: `neighborsPtr` - búsqueda básica con octree basado en punteros. `neighbors` - búsqueda básica con octree lineal. `neighborsPrune` - optimizada, con poda de octantes. `neighborsStruct` - optimizada, usando rangos de índices. `neighborsApprox` - búsqueda aproximada (lower y upper bounds). `neighborsUnibn` - búsqueda con unibnOctree. `neighborsPCLKD` - búsqueda con KD-tree de PCL. `neighborsPCLOct` - búsqueda con Octree de PCL.
`-e`, `--encodings`	Codificaciones SFC para reordenar la nube antes de la búsqueda (separadas por comas o `all`). Por defecto: `all`. Valores posibles: `none`, `mort`, `hilb`.

Cuadro B.2: Opciones adicionales del programa

Opción	Descripción
`--debug`	Activa el modo depuración (mide tiempos de construcción del octree y codificación).
`--check`	Verifica los resultados (opción antigua, puede no funcionar; se puede usar μ para validar).
`--no-warmup`	Desactiva la fase de calentamiento inicial.
`--approx-tol`	Tolerancias para las búsquedas aproximadas (en porcentajes, por ejemplo: $10.0, 50.0, 75.0, 100.0$).
`--num-threads`	Lista de número de hilos para pruebas de escalabilidad (ej. $1, 2, 4, 8, 16, 32$). Por defecto, OpenMP usa el número máximo.
`--sequential`	Hace que el conjunto de búsqueda sea secuencial en vez de aleatorio (más rápido). Si se usa `-s all`, esto se activa automáticamente. La elección del primer punto de búsqueda sigue siendo aleatoria, en el rango $[0, N-s)$, donde s es el número de búsquedas a realizar.
`--max-leaf`	Número máximo de puntos por hoja del octree (por defecto: 128). No aplica a octrees de PCL.
`--pcl-oct-resolution`	Tamaño mínimo de octantes para subdivisión en el Octree de PCL.

Apéndice C

Licencia

Esta obra está bajo una licencia Creative Commons «Atribución-NoComercial-CompartirIgual 3.0 No portada».



Bibliografía

[1] Ciprian Apetrei. «Fast and Simple Agglomerative LBVH Construction». En: *Computer Graphics and Visual Computing (CGVC)*. Ed. por Rita Borgo y Wen Tang. The Eurographics Association, 2014. ISBN: 978-3-905674-70-5. DOI: /10.2312/cgvc.20141206.

[2] Tetsuo Asano et al. «Space-filling curves and their use in the design of geometric data structures». En: *Theoretical Computer Science* 181.1 (1997), págs. 3-15.

[3] J. Baert. *Libmorton: Biblioteca C++ para Codificación/Decodificación de Morton*. https://github.com/Forceflow/libmorton. Accedido: 3 de julio de 2025. 2018.

[4] Jens Behley, Volker Steinhage y Armin B Cremers. «Efficient radius neighbor search in three-dimensional point clouds». En: *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE. 2015, págs. 3625-3630.

[5] Jon Louis Bentley. «Multidimensional binary search trees used for associative searching». En: *Communications of the ACM* 18.9 (1975), págs. 509-517.

[6] Christian Böhm, Martin Perdacher y Claudia Plant. «A novel hilbert curve for cache-locality preserving loops». En: *IEEE Transactions on Big Data* 7.2 (2018), págs. 241-254.

[7] Benjamin Brede et al. *Speulderbos terrestrial (tls) and unmanned aerial vehicle laser scanning (uav-ls) 2017*. 2020.

[8] X. Cao et al. *Stacked autoencoder-based compression of point cloud geometry*. 2025. DOI: 10.21203/rs.3.rs-5665971/v1.

[9] Rener Castro et al. «Statistical optimization of octree searches». En: *Computer Graphics Forum*. Wiley Online Library. 2008, págs. 1557-1566.

[10] CCCL Development Team. *CCCL: CUDA C++ Core Libraries*. 2023. URL: https://github.com/NVIDIA/cccl.

[11] J. Chen, L. Yu y W. Wang. «Hilbert space filling curve based scan-order for point cloud attribute compression». En: *IEEE Transactions on Image Processing* 31 (2022), págs. 4609-4621. DOI: 10.1109/tip.2022.3186532.

[12] Wanli Chen et al. «Efficient Point Cloud Analysis Using Hilbert Curve». En: *Computer Vision – ECCV 2022*. Ed. por Shai Avidan et al. Cham: Springer Nature Switzerland, 2022, págs. 730-747. ISBN: 978-3-031-20086-1.

[13] Pablo Díaz Viñambres. *octrees-benchmark: Repositorio del trabajo*. 2025. URL: https://github.com/derivada/octrees-benchmark/tree/tfg.

[14] Miguel Yermo García. «HPC solutions for ALS point cloud processing in pathfinding and powerline detection and characterization». Tesis doct. Universidade de Santiago de Compostela, 2024.

[15] Xuefeng Guan, Peter Van Oosterom y Bo Cheng. «A Parallel N-Dimensional Space-Filling Curve Library and Its Application in Massive Point Cloud Management». En: *ISPRS International Journal of Geo-Information* 7.8 (2018). ISSN: 2220-9964. DOI: 10.3390/ijgi7080327. URL: https://www.mdpi.com/2220-9964/7/8/327.

[16] Antonin Guttman. «R-trees: A dynamic index structure for spatial searching». En: *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*. 1984, págs. 47-57.

[17] Timo Hackel et al. «SEMANTIC3D.NET: A new large-scale point cloud classification benchmark». En: *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*. Vol. IV-1-W1. 2017, págs. 91-98.

[18] Herman Haverkort. «How many three-dimensional Hilbert curves are there?» En: *arXiv preprint arXiv:1610.00155* (2016).

[19] Rama C Hoetzlein. «Fast fixed-radius nearest neighbors: interactive million-particle fluids». En: *GPU Technology Conference*. Vol. 18. 2014, pág. 2.

[20] M. Isenburg. *LAStools-efficient tools for LiDAR processing*. https://github.com/LAStools/LAStools/. Accedido: 3 de julio de 2025. 2012.

[21] Martin Isenburg. «LASzip: lossless compression of LiDAR data». En: *Photogrammetric engineering and remote sensing* 79.2 (2013), págs. 209-217.

[22] Tero Karras. «Maximizing parallelism in the construction of BVHs, octrees, and k-d trees». En: *Proceedings of the Fourth ACM SIGGRAPH/Eurographics Conference on High-Performance Graphics*. 2012, págs. 33-37.

[23] Sebastian Keller et al. «Cornerstone: Octree Construction Algorithms for Scalable Particle Simulations». En: *Proceedings of the Platform for Advanced Scientific Computing Conference*. PASC '23. ACM, jun. de 2023, págs. 1-10. DOI: 10.1145/3592979.3593417. URL: http://dx.doi.org/10.1145/3592979.3593417.

[24] Warren M Lam y JH Shapiro. «A class of fast algorithms for the Peano-Hilbert space-filling curve». En: *Proceedings of 1st International Conference on Image Processing*. Vol. 1. IEEE. 1994, págs. 638-641.

[25] Yohei Miki y Masayuki Umemura. «GOTHIC: Gravitational oct-tree code accelerated by hierarchical time step controlling». En: *New astronomy* 52 (2017), págs. 65-81.

[26] Guy M Morton. *A computer oriented geodetic data base and a new technique in file sequencing*. IBM, Ottawa, Canada. 1966.

[27] Marius Muja y David G Lowe. «Fast approximate nearest neighbors with automatic algorithm configuration.» En: *VISAPP (1)* 2.331-340 (2009), pág. 2.

[28] OpenMP Architecture Review Board. *The OpenMP API Specifications for Parallel Programming*. `https://www.openmp.org/specifications/`. Accedido: 3 de julio de 2025.

[29] M. Pavlović et al. «Dictionary compression in point cloud data management». En: *ACM Transactions on Spatial Algorithms and Systems* 5 (1 2019), págs. 1-25. DOI: `10.1145/3299770`.

[30] American Society for Photogrammetry y Remote Sensing (ASPRS). *LAS Specification Version 1.4 - R13*. `https://www.asprs.org/wp-content/uploads/2010/12/LAS_1_4_r13.pdf`. Accedido: 3 de julio de 2025. 2013.

[31] Xavier Roynard, Jean-Emmanuel Deschaud y François Goulette. «Paris-Lille-3D: A large and high-quality ground-truth urban point cloud dataset for automatic segmentation and classification». En: *The International Journal of Robotics Research* 37.6 (2018), págs. 545-557.

[32] Radu Bogdan Rusu y Steve Cousins. «3D is here: Point Cloud Library (PCL)». En: *IEEE International Conference on Robotics and Automation (ICRA)*. Shanghai, China: IEEE, mayo de 2011.

[33] Y. Shao et al. «Texture-guided graph transform optimization for point cloud attribute compression». En: *Applied Sciences* 14 (10 2024), pág. 4094. DOI: `10.3390/app14104094`.

[34] Nina Varney, Vijayan K Asari y Quinn Graehling. «DALES: A Large-scale Aerial LiDAR Data Set for Semantic Segmentation». En: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 2020, págs. 186-187.

[35] Wei Wang et al. «A Hybrid Spatial Indexing Structure of Massive Point Cloud Based on Octree and 3D R*-Tree». En: *Applied Sciences* 11.20 (2021). ISSN: 2076-3417. DOI: `10.3390/app11209581`. URL: `https://www.mdpi.com/2076-3417/11/20/9581`.

[36] Michael S Warren y John K Salmon. «A parallel hashed oct-tree n-body algorithm». En: *Proceedings of the 1993 ACM/IEEE conference on Supercomputing*. 1993, págs. 12-21.

[37] Y. Yang et al. «Integrating nosql, hilbert curve, and r*-tree to efficiently manage mobile lidar point cloud data». En: *ISPRS International Journal of Geo-Information* 13 (7 2024), pág. 253. DOI: 10.3390/ijgi13070253.